Lecture 7. DevOps Concept and Practice

한국공학대학교
TECH UNIVERSITY OF KOREA

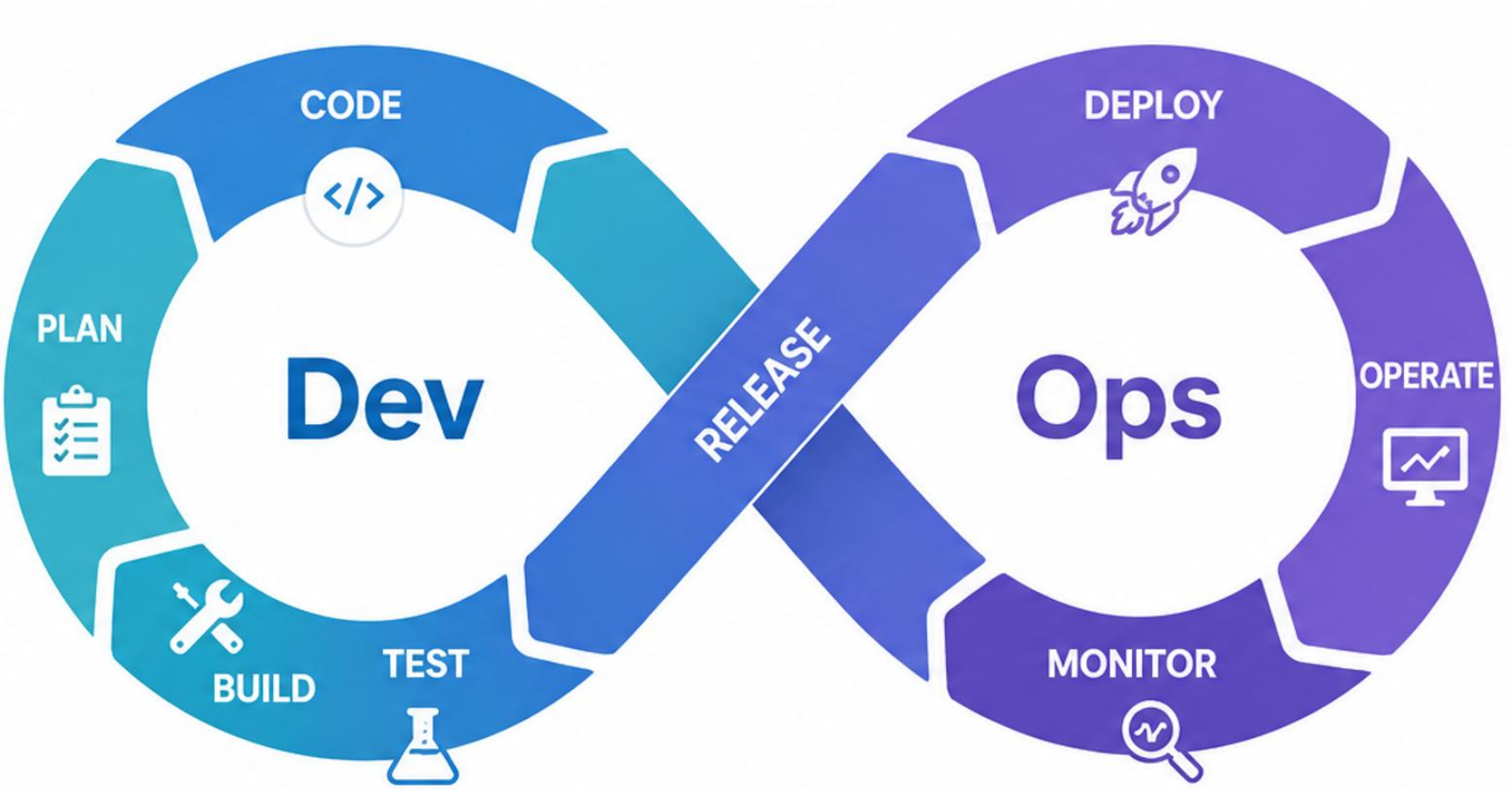
Lee Youngkon

DevOps Background

- Existing Development Method
  - Development team → code writing
  - Operations Team → Service Operations
- problem
  - Slow deployment
  - Separation of development and operational responsibilities
  - Failure Response Delay
  - Environmental inconsistency

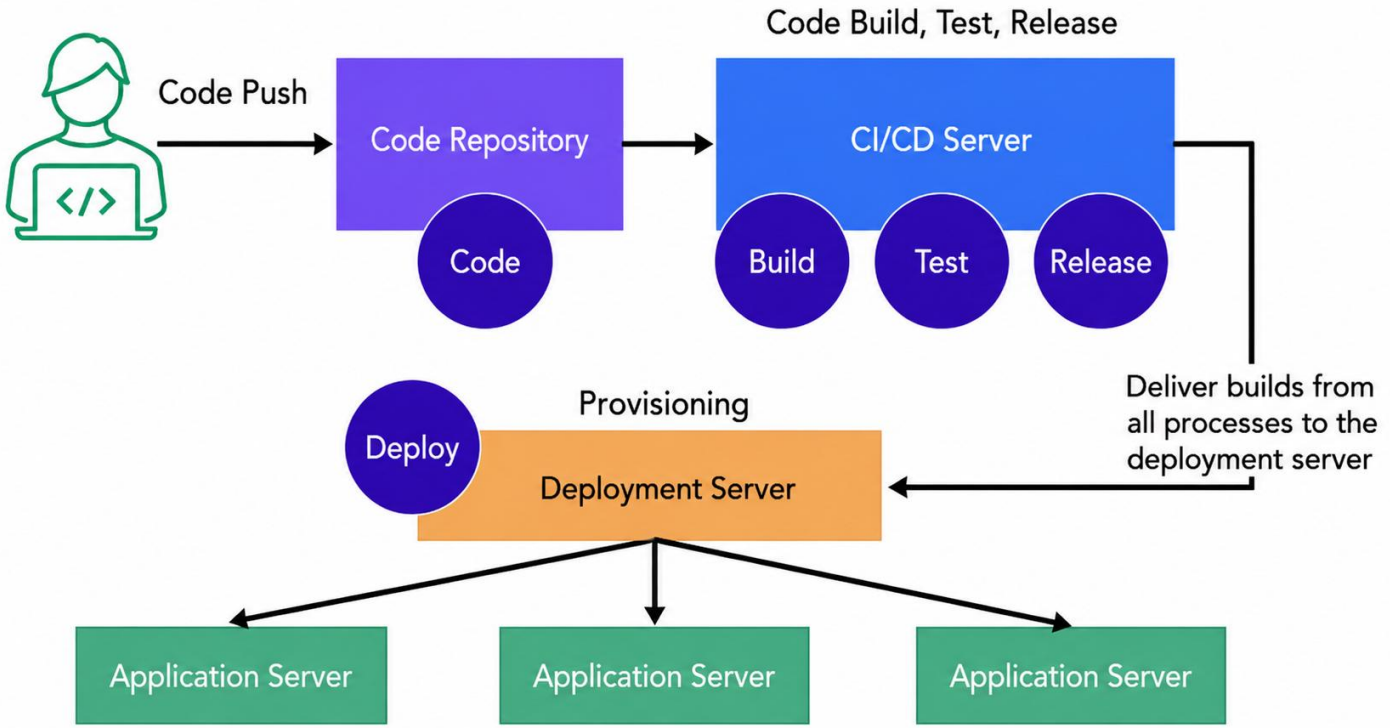
- DevOps Goals
  - Development + Operational Integration
  - Automation-based deployment
  - Fast service delivery
  - stable operation
  - Maintaining Security

DevOps Concept



DevOps Components

- Continuous Integration (CI)
- Continuous Delivery (CD)
- Continuous Monitoring
- Automation
- Collaboration



DevOps Concept: DevOps 10 Principles

1. the prohibition of reckless expansion of automation

- CI/CD, IaC, deployment automation required
- Unverified automation can spread failures on a large scale
- The following must be provided
  - Test Automation
  - Approval Process
  - Rollback strategy
  - Blue/Green Deployment
  - Deploy Canary

2. Internalize Security (DevSecOps)

- Don't hardcode secret keys, DB passwords, certificates
- Compliance with
  - Docker Secret / Kubernetes Secret
  - Vault
  - Scan for Image Vulnerabilities
  - SAST/DAST
  - RBAC Minimum Privilege Principle

❖ Static Application Security Testing (SAST): A method of statically analyzing source code, binary, or build results without running them to find security vulnerabilities. ex. SonarQube
❖ Dynamic Application Security Testing (DAST): How to test running applications (Web/API) from the perspective of external attackers. ex. Buff Suit

3. Do Not Deploy Without Monitoring

- "Successful distribution" ≠ "Normal service"
- Requirements:
  - Collect Logs (ELK, Loki)
  - Metric (Prometheus)
  - Tracing (Jaeger)
  - Alert manager

4. Maintain environmental consistency

- Minimize development/testing/operational environmental differences
- Docker/Kubernetes/IaC(Terraform) active utilization

5. Prevent data loss

- Most dangerous when changing the DB
- Compliance with
  - Managing Backup Schema Versions
  - Validation of migration
  - PV/PVC Strategy
  - Rehearsal

6. Management of inter-service dependencies

- API version management
- Circuit Breaker
- Retry Policy
- Timeout
- Message duplication (Idempotency)

❖ MTTR (Mean Time To Recovery)
❖ Change Failure Rate: Percentage of deployment changes causing failures/rollbacks/hotfixes

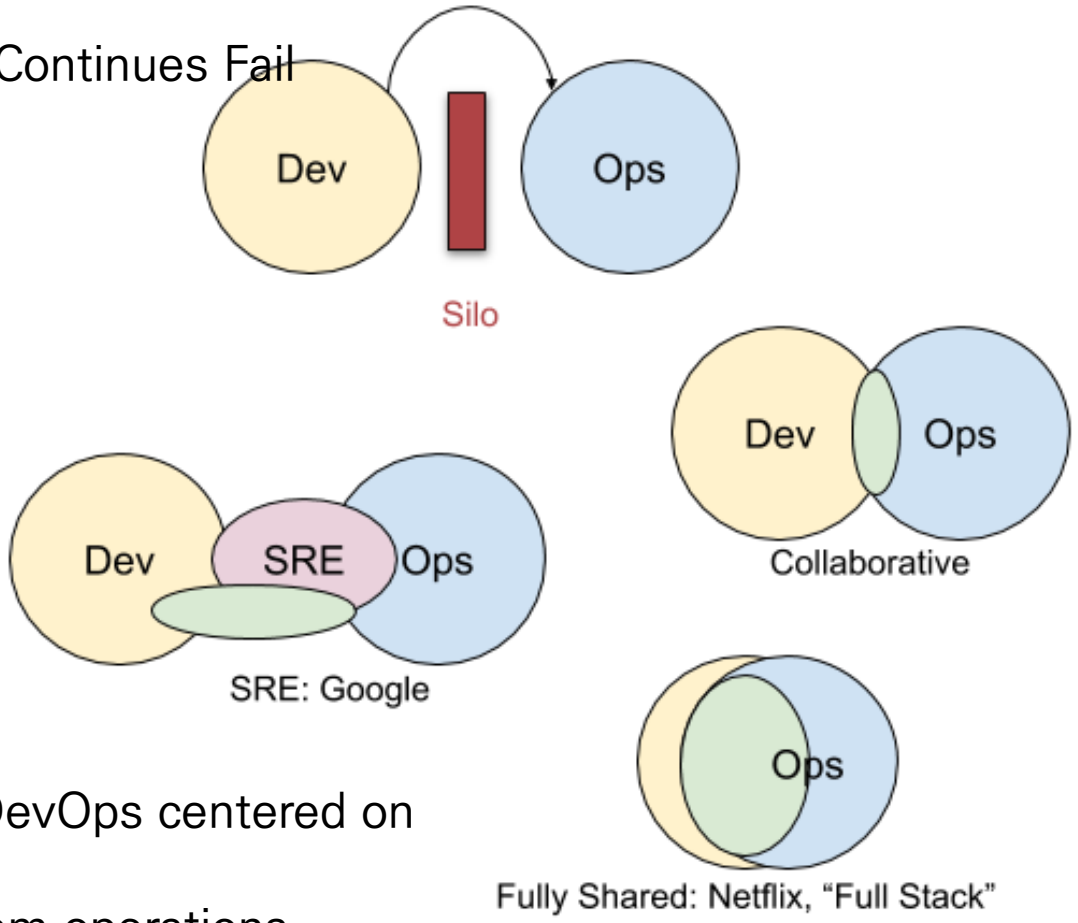
7. Reliability over deployment speed

- Frequent deployment itself is not the goal
- KPI
  - MTTR
  - Deployment Frequency
  - Change Failure Rate

DORA Metrics(DevOps Research and Assessment)

8. organizational culture issues

- Dev vs Ops Separation Culture Continues Fail
- Key points:
  - joint responsibility
  - collaboration
  - Standardization
  - documentation



❖ SRE(Site Reliability Engineering)
- Engineering framework to deploy DevOps centered on operational stability
- Responsibilities for managing system operations, availability, scalability, and recoverability
- Resolve operations (Ops) with code and automation, not by hand

9. Cost Management

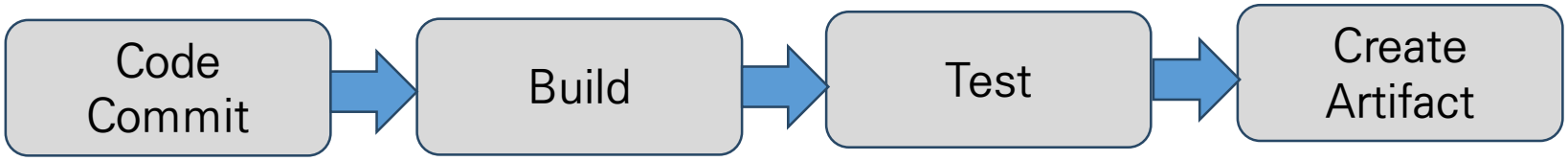
- Cloud/K8s is expensive when it is used carelessly
- precautions
  - Excessive resource request
  - Unused Pod
  - Over-save logs
  - GPU/storage waste

10. Disaster Recovery Plan

- Disaster must happen
- Recovery plan:
  - DR Scenario, Multiple AZs, Backup, Restore Test, Operational Manual

Continuous Integration (CI)

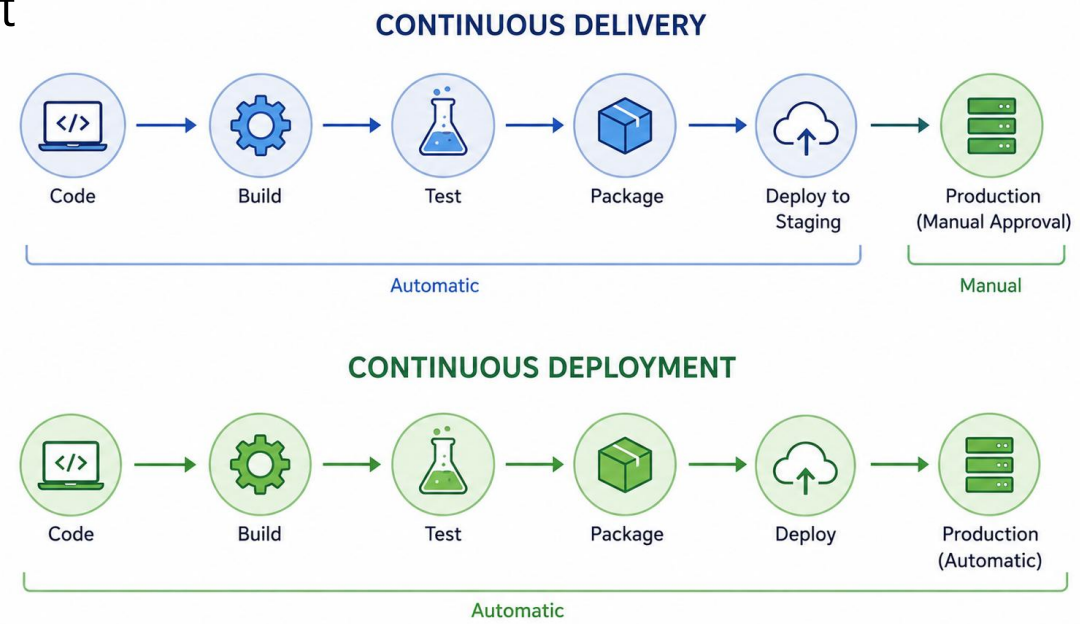
- Automatically build and test developer-generated code



- Representative Tools: Jenkins, GitHub Action, GitLab CI, Circle CI

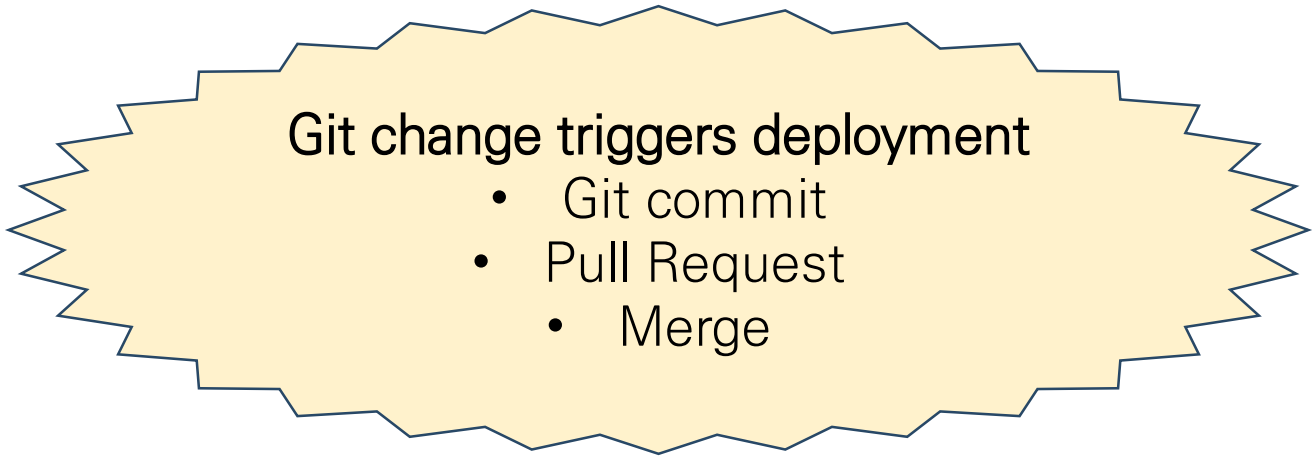
Continuous Delivery / Deployment

- Continuous Delivery
  - Auto Build + Test
  - Manually Approve Deployment
- Continuous Deployment
  - Automatic build
  - Automatic test
  - Automated Deployment



Define GitOps

- Operational approach to managing infrastructure and applications using Git as a single source of truth
  - Git = System State = Baseline document for current operational state



GitOps Core Principles

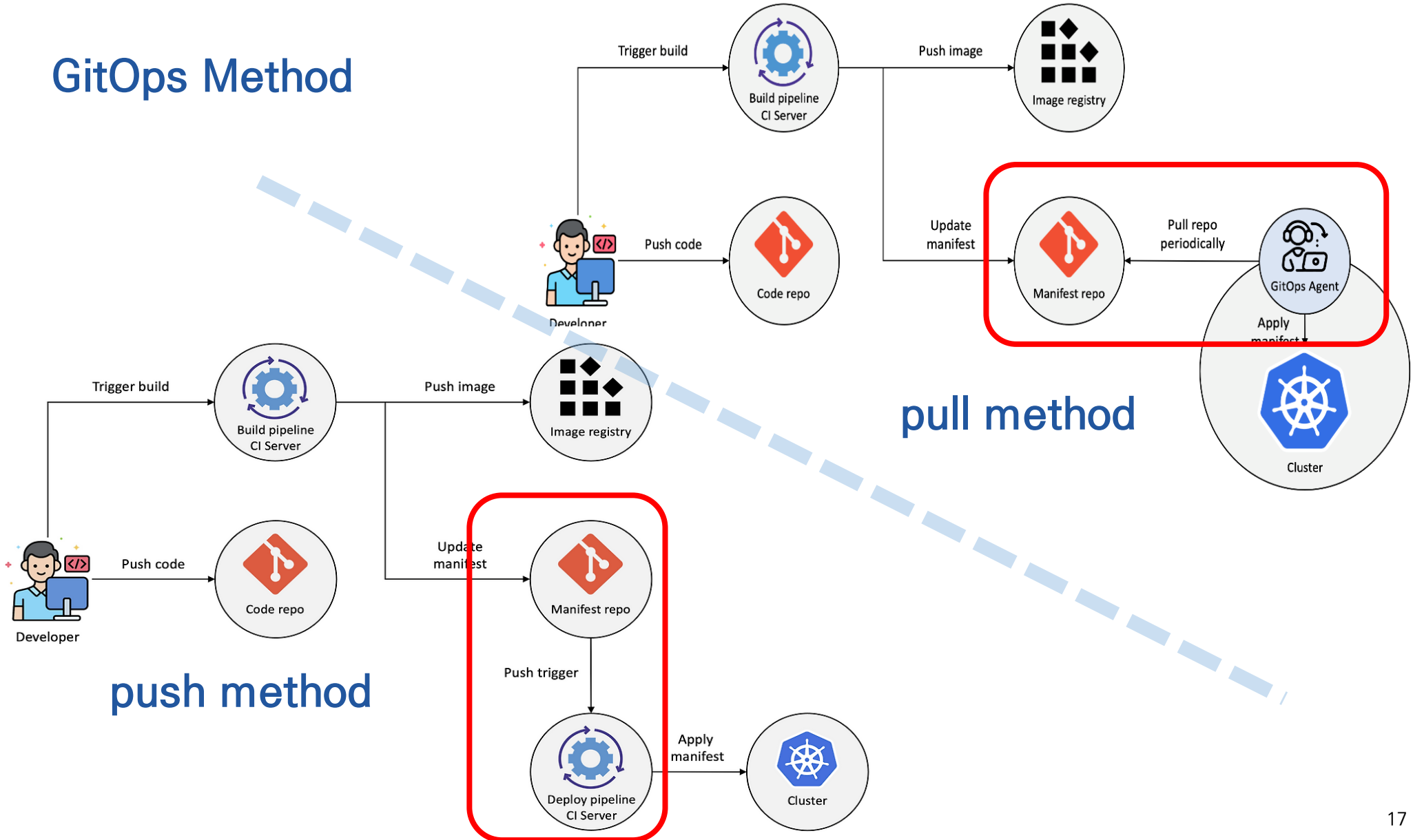
- declarative composition
  - Infrastructure as Code
  - Kubernetes YAML
  - Helm
- Git Single Source
  - Save all states Git
  - deployment.yaml
  - service.yaml
- automatic synchronization
  - Git Change → System Reflections
- Pull method
  - System monitors Git
  - Cluster → Git

GitOps Core Tools

Way	Optional/ Require	Role	note
Kubernetes YAML	Optional	Define the most basic deployments	
Helm Chart	Optional	Template-based deployment	Complex service packaging, variable management, version management
Kustomize	Optional	Overlay by dev/stage/prod environment	Maintain YAML source. Good readability. Simple environmental separation clear
Terraform	Optional	Create Cloud/Infrastructure Resources	Utilize when running IaC
Argo CD / Flux	Required	Synchronize Git and Cluster States	Flux takes advantage of declarative automation and image update automation more strongly than UI

GitOps Concepts

GitOps Method



GitOps Components

- Git Repository
- CI Pipeline
- Container Registry
- GitOps Controller
- Kubernetes Cluster

Git Repository

- Structure for storing project files and all changes
- Simple File Folder + Change History DB

Function	Description
Version Control	Save Change History
Branching	Quarter by function
Merge	code integration
Rollback	Recovery of previous state
Collaboration	team collaboration
Audit	Track who changed what

Git Repository

category	GitHub	GitLab
SaaS	Industry-leading SaaS platform	Mature SaaS Platform
On-premise	Enterprise requires separate deployment	Full Self-host Support
CI/CD	Extensions based on GitHub Actions	Basic Integrated Platform
DevSecOps Integration	Center for external tool linkage	Basic integration of security features
UI/UX	User-friendly	Feature-driven
open-source ecosystem	the largest ecosystem	a powerful ecosystem of its own
Corporate Security Control	outstanding	Enterprise-class control
Cost	SaaS-centric, Enterprise costs high	Self-host based. cost-effectiveness

CI Pipeline

- Automated verification process that automatically builds, tests, quality checks, security checks, and more whenever developers push code changes to Git
- Fast and secure integration of code changes
  - Code crash early detection
  - Maintaining Quality
  - Ensuring deployment readiness
  - Automated Reliability Enhancements

Git Push
→ GitHub Actions
→ Maven Build
→ JUnit
→ SonarQube
→ Docker Build
→ Harbor Push

Container Registry

- Central repository for storing, versioning, and deploying Docker/OCI container images
- After the container build, the container image exists only on the local PC
- You must be available in the following environments to deploy your production environment
  - Kubernetes
  - Another server
  - CI/CD Development Team

Requires a repository accessible to all

☞ Container Registry use

Container Registry Features

Function	Description
Image Storage	Save Image
Version Tagging	v1.0.1 etc
Access Control	User/Team Rights
Vulnerability Scan	Security vulnerability check
Replication	Multiple environment replication
Signature	Image Signing
Audit	Track who distributed it

Container Registry Comparison

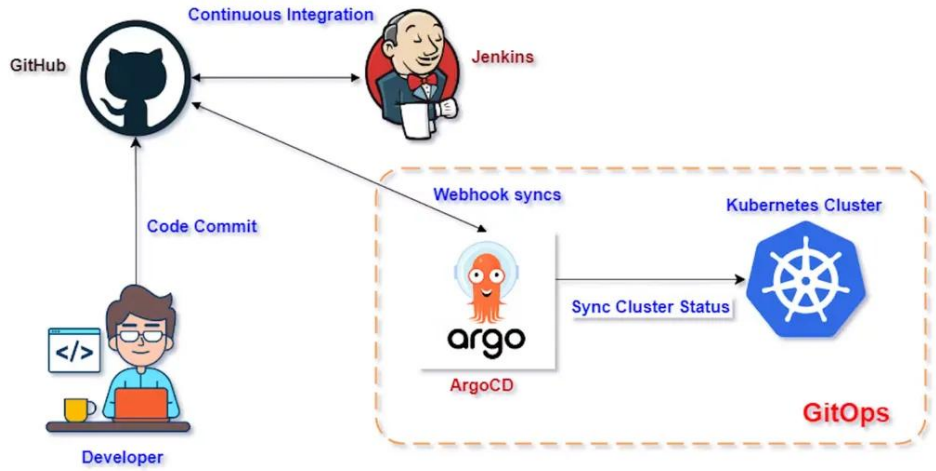
category	Docker Hub	Harbor
Operation type	SaaS-based	Self-host / Private Registry
a public image	Global Largest	restrictive
Private Registry	Support	Enterprise-grade
On-premise	Enterprise-centric	Full support
security control	Standard level	Advanced Policy Control
Scan for vulnerabilities	Partial Support	Basic Interior
Image Signing	partial support	Policy-based support
RBAC	Managing Default Privileges	fine-grained access control
Replication	restrictive	Multi-site support
Speed	Internet environmental dependence	internal network optimization
Cost control	Subscription-based	Optimizing your own operations
Enterprise DevOps Conformity	a mid-range level	Best for big business/on-premises

GitOps Architecture

GitOps Controller

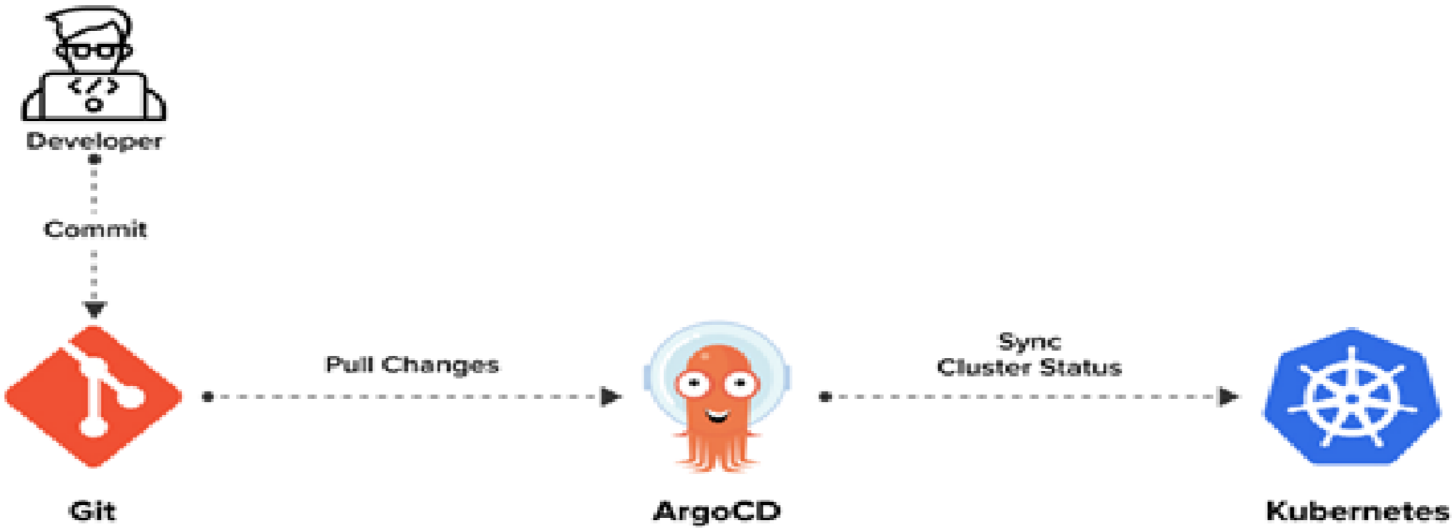
- Operational automation engine that continuously compares and automatically matches actual Kubernetes/infrastructure state with desired system state defined in Git repository

- Actual Cluster State ≠Git State → Automatic Correction
- GitOps Controller = "Automatic Deployment + Continuous Health Monitoring + Drift Recovery System"
- Representative Tools: Argo CD, Flux



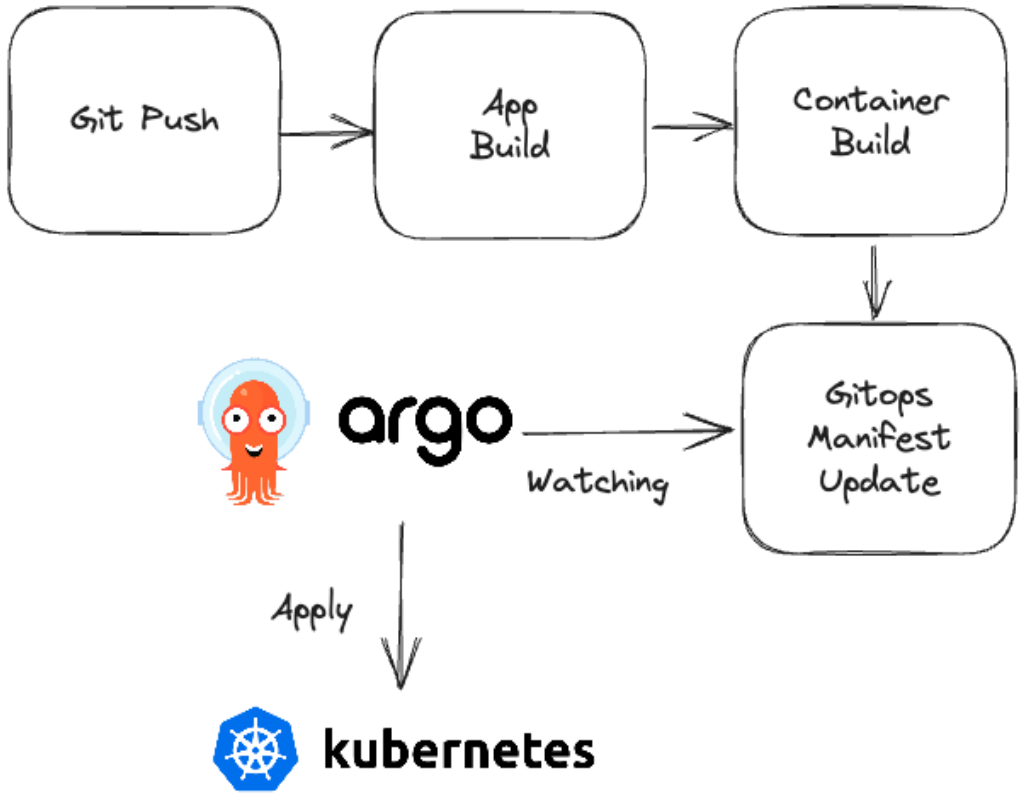
ArgoCD: Git Ops Deployment Tool for Kubernetes

- Automate Kubernetes Deployment
- Git health monitoring
- automatic synchronization
- Rollback Support

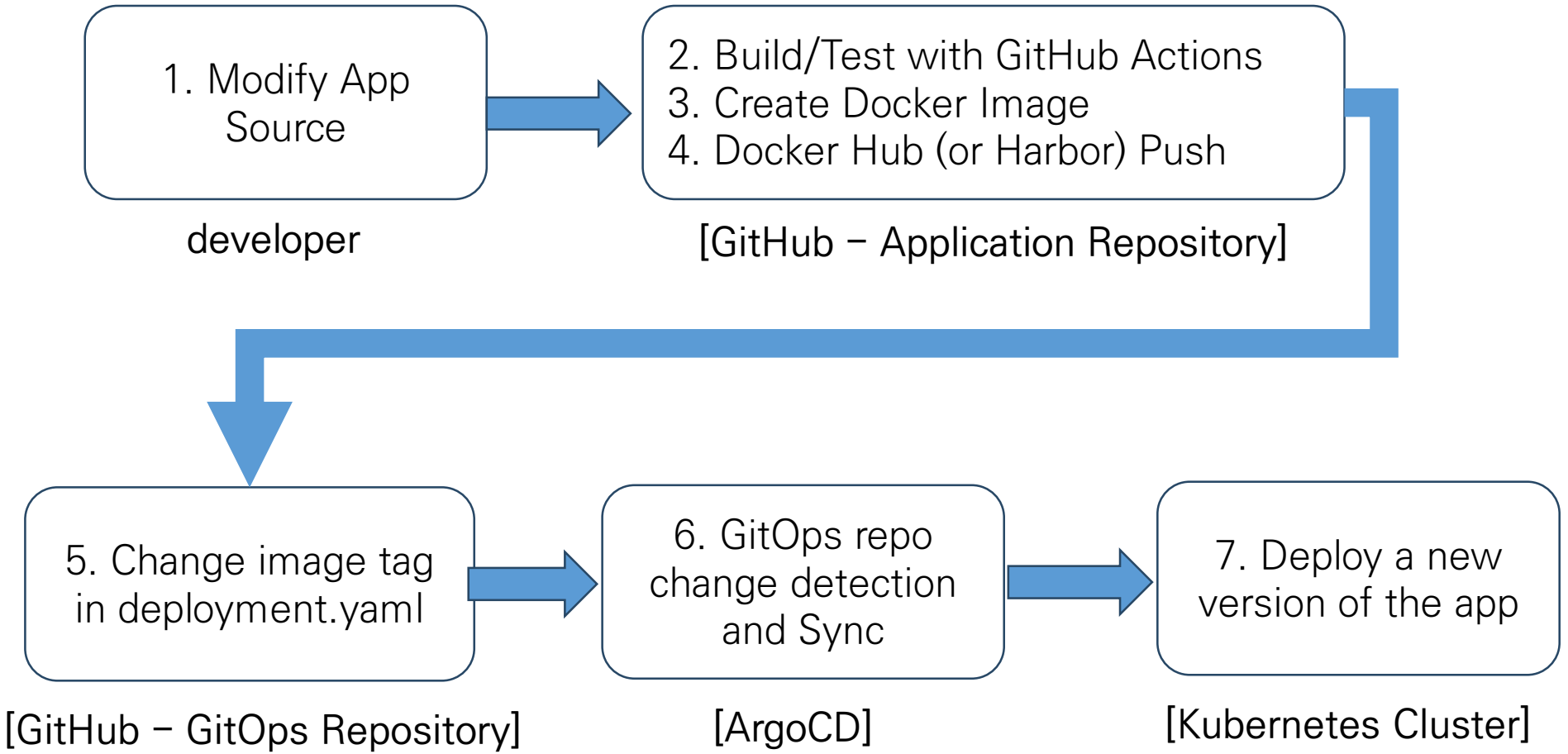


ArgoCD Operation Course

- Git Repository check
  - deployment.yaml
  - service.yaml
- Cluster status comparison
  - Desired State
  - Actual State
- Occurrence of difference
  - Synchronize
- Deploying Kubernetes



Practice Process



Practice Process

③ Change the manifest

② container image push

④ Manifest change detection

① commit & push

⑤ Apply changes to the cluster

kubernetes cluster

Application Repository

Configuration

```
book-api
├── src
│   ├── main
│   │   ├── java/com/example/bookapi
│   │   │   ├── BookApiApplication.java
│   │   │   ├── controller/BookController.java
│   │   │   ├── model/Book.java
│   │   │   └── service/BookService.java
│   │   └── resources
│   │       └── application.yml
├── pom.xml
├── Dockerfile
└── .github
    └── workflows
        └── ci.yaml
```

Role

- Java Code Management
- Maven Build Test
- Create Docker Image
- Registry Push

GitOps Repository

Configuration

```
book-api-gitops
├── base
│   ├── deployment.yaml
│   ├── service.yaml
│   └── kustomization.yaml
└── overlays
    └── dev
        ├── namespace.yaml
        └── kustomization.yaml
```

Role

- Managing Kubernetes manifest
- Managing the deployment version (image tag)
- Managing settings by environment (dev/prod)
- Reference repository that ArgoCD refers to

CI/CD for GitOps Repository and App Repository Same or Separation

same

① Git Push

② Build

③ Docker Hub Push

④ Modify k8s/deployment.yaml image tag

⑤ Same repo commit

⑥ ArgoCD Sync

Separation

① App Repo Push

② Build

③ Docker Hub Push

④ Modify GitOps Repo deployment.yaml

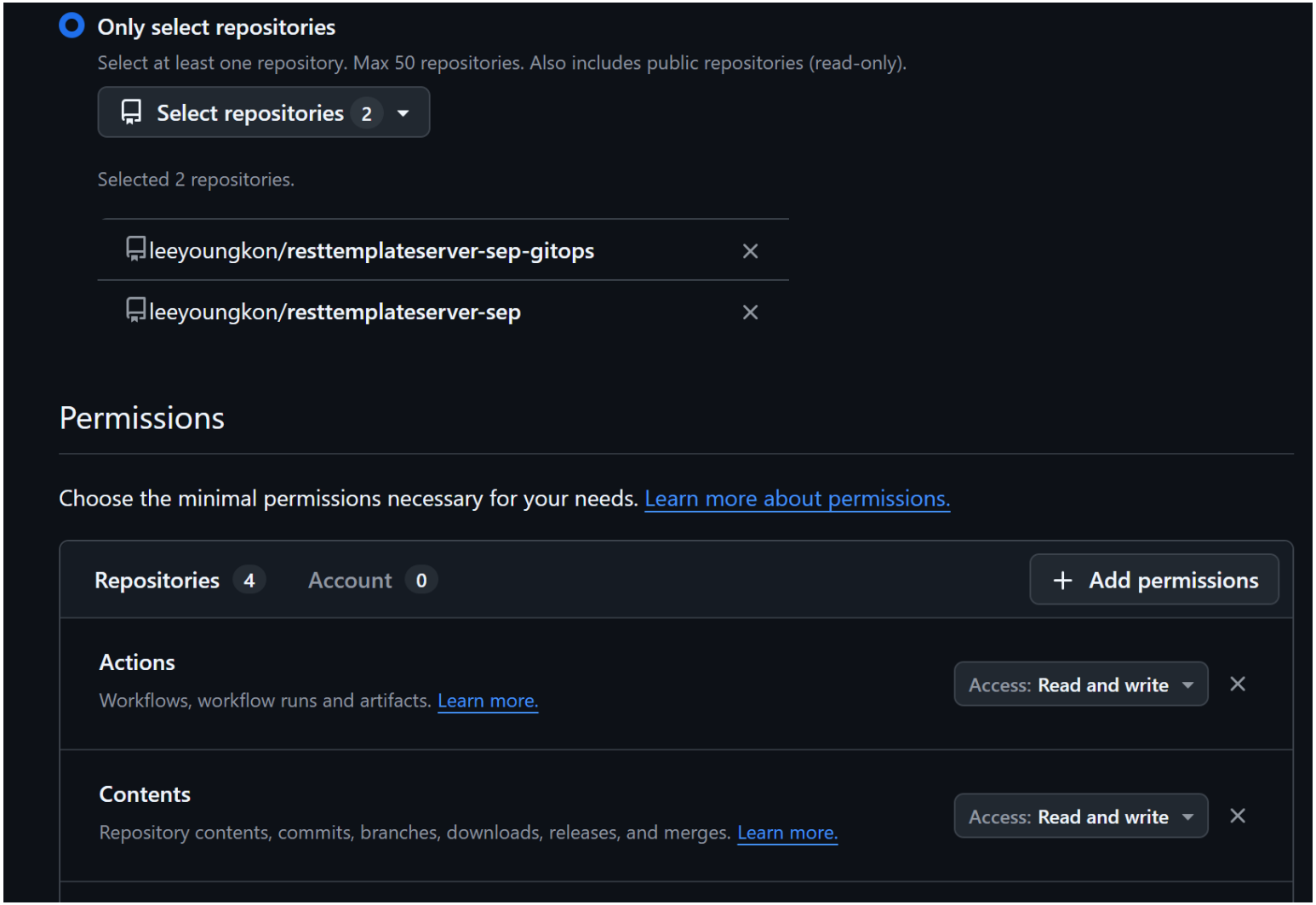
⑤ GitOps Repo commit

⑥ ArgoCD Sync

When separating the GitOps repository from the App repository

- GITOPS_TOKEN must be registered in App repo.
- That way, the app repo can have permission to modify the contents of the GitOps repo.
- GITHUB_TOKEN is issued according to the procedure below, and it is registered as GITOPS_TOKEN in App repo's secret.
  - ① GitHub login
  - ② Settings
  - ③ Developer settings
  - ④ Personal access tokens
  - ⑤ Fine-grained tokens
  - ⑥ Generate new token

When separating the GitOps repository from the App repository



When separating the GitOps repository from the App repository

HEALTH	Healthy
LINKS	
IMAGES	210.90.24.172:8080/camit/yklee2002/resttemplateserver-sep:latest mysql:8.0

SOURCE EDIT

REPO URL	https://github.com/leeyoungkon/resttemplateserver-sep-gitops.git
TARGET REVISION	HEAD
PATH	overlays/dev

CI.yml

Using the CI Server in Docker hub

```
name: CI/CD Pipeline

on:
  push:
    branches:
      - main

env:
  DOCKERHUB_USERNAME:
${{ secrets.DOCKERHUB_USERNAME }}
  IMAGE_NAME: resttemplateserver-hub

jobs:
  build-and-push:
    name: Build & Push to Docker Hub
    runs-on: ubuntu-latest

    outputs:
      image_tag: ${{ steps.meta.outputs.tag }}

    steps:
      - name: Checkout source
        uses: actions/checkout@v4
```

CI.yml

```
- name: Set up JDK 17
  uses: actions/setup-java@v4
  with:
    java-version: '17'
    distribution: 'temurin'
    cache: maven

- name: Grant execute permission for Maven Wrapper
  run: chmod +x ./mvnw

- name: Build with Maven
  run: ./mvnw clean package -DskipTests

- name: Generate image tag
  id: meta
  run: echo "tag=${{ github.sha }}" >> $GITHUB_OUTPUT

- name: Log in to Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${{ secrets.DOCKERHUB_USERNAME }}
    password: ${{ secrets.DOCKERHUB_TOKEN }}
```

CI.yml

```
- name: Build and push Docker image
      uses: docker/build-push-action@v5
      with:
        context: .
        push: true
        tags: |
          ${{ env.DOCKERHUB_USERNAME }}/${{ env.IMAGE_NAME }}:
          ${{ env.DOCKERHUB_USERNAME }}/${{ env.IMAGE_NAME }}:latest

 update-manifest:
   name: Update K8s Manifest for ArgoCD
   runs-on: ubuntu-latest
   needs: build-and-push
```

CI.yml

```
steps:
    - name: Checkout source
      uses: actions/checkout@v4
      with:
        token: ${{ secrets.GITHUB_TOKEN }}

    - name: Update image tag in k8s.yaml
      run: |
        NEW_IMAGE="${{ env.DOCKERHUB_USERNAME }}/${{ env.IMAGE_NAME }}:---"
        sed -i "s|image: .*${{ env.IMAGE_NAME }}.*|image: ${NEW_IMAGE}|g" k8s.yaml

    - name: Commit and push manifest update
      run: |
        git config user.name "github-actions[bot]"
        git config user.email "github-actions[bot]@users.noreply.github.com"
        git add k8s.yaml
        git diff --staged --quiet || git commit -m "ci: update image to ---}}"
        git push
```

Replace image tags for k8s.yaml

Commit modifying the manifest file

Reasons for Image Tag Replacement by CI.yml (SHA)

- If you only use the last, latest = mutable
  - Content can be changed with the same tag –〉 Difficult to trace
  - Low Reproducibility
  - Difficulty in recovery

Way	uniqueness	Rollback	traceability
latest	low	difficulty	low
SHA	high	Easy	high

Role classification

CI Role

- In Application reply:source checkout
- Maven build/test
- Docker image build
- Docker registry push
- GitOps repo checkout
- Reflect new image tag
- GitOps repo commit/push

Role of ArgoCD

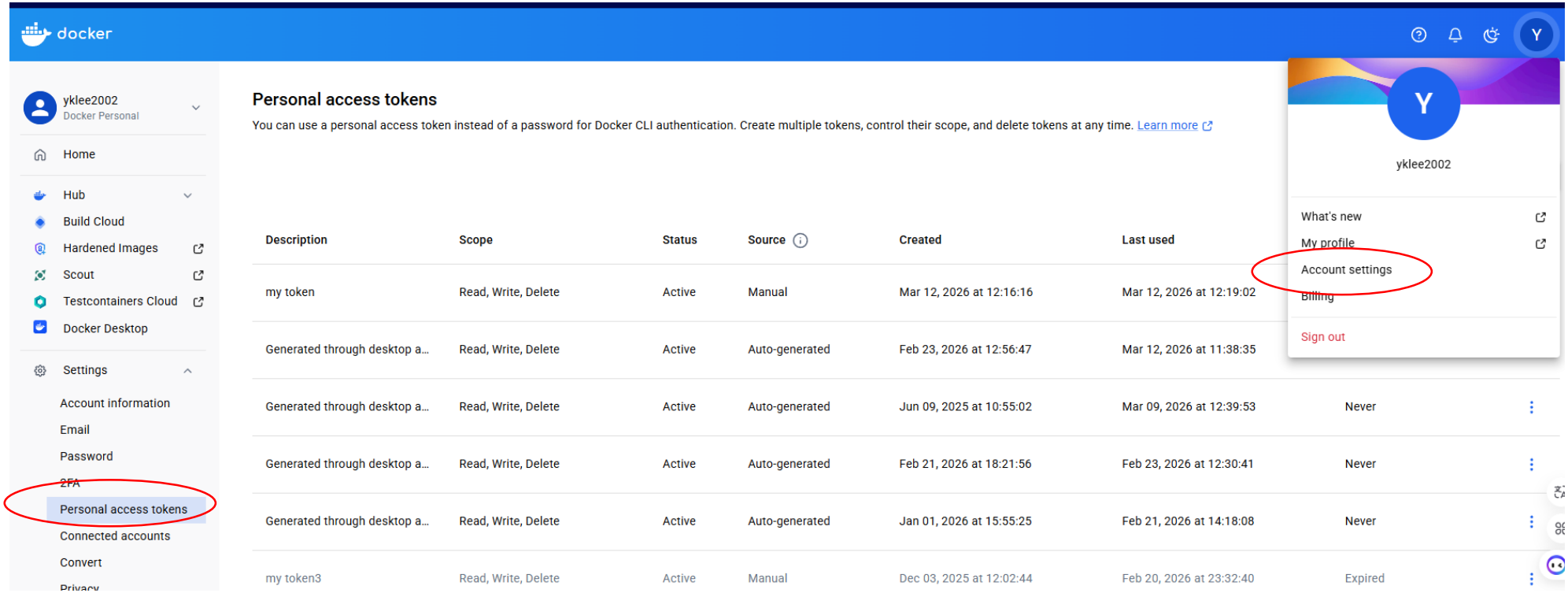
- Monitor GitOps repo changes
- Compare to desired state and actual state
- sync when detecting difference
- Reflect Kubernetes Deployment

ArgoCD.yml

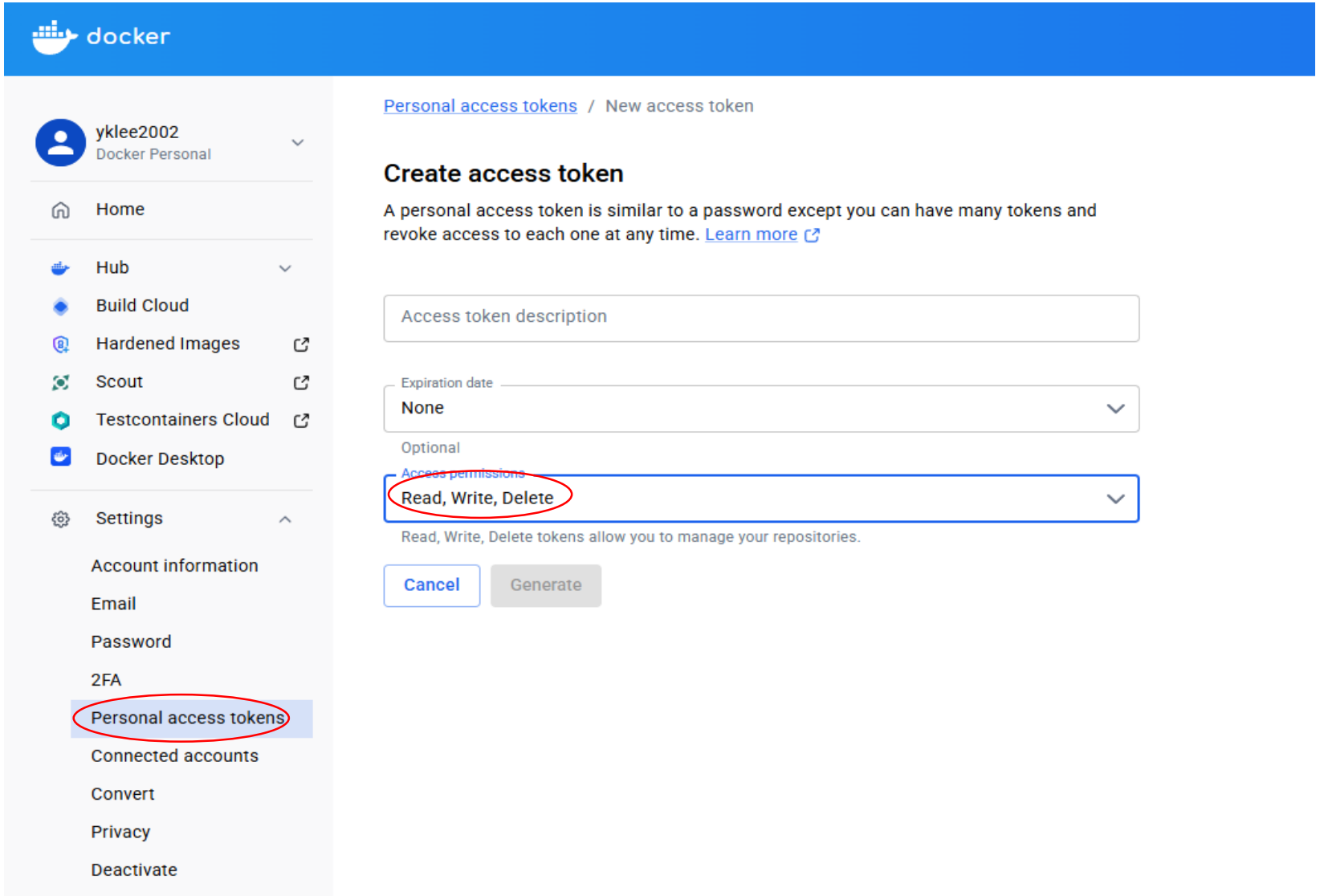
by tool
automatic
generation

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: book-api-dev
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/your-id/book-api-gitops.git
    targetRevision: HEAD
    path: overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: book-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

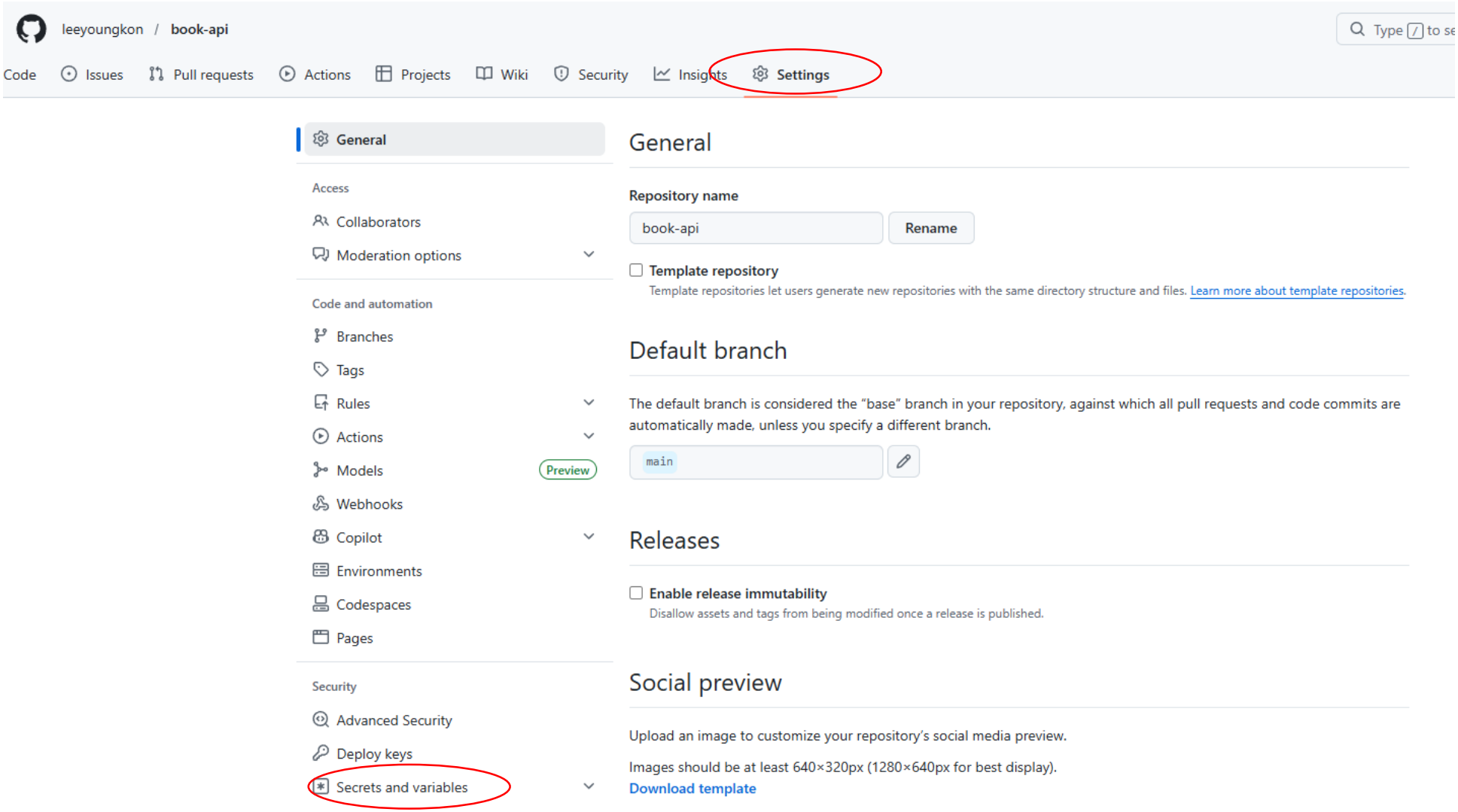
Preparation at Docker hub ⇒ generate token



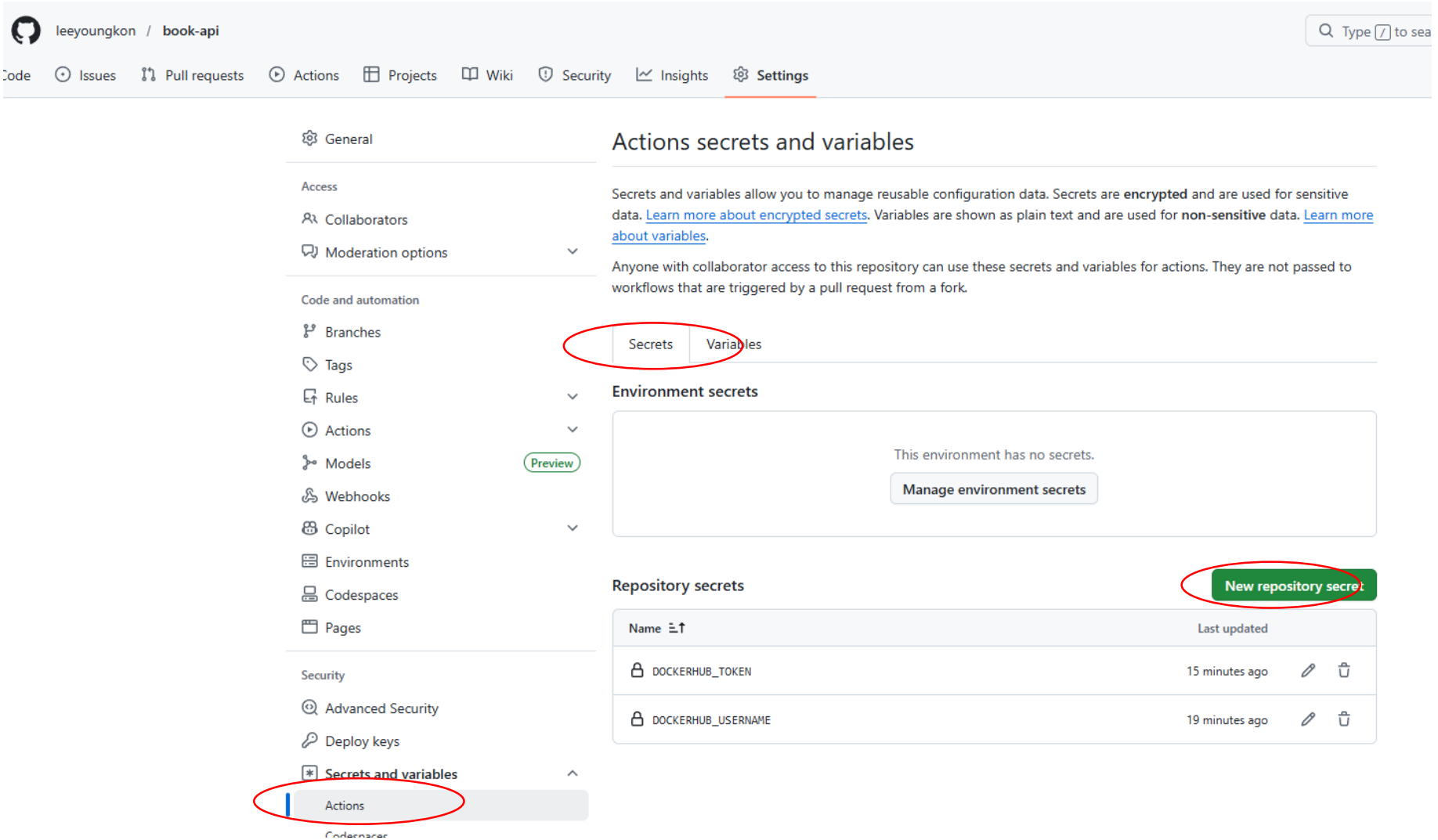
Preparation at Docker hub ⇒ generate token



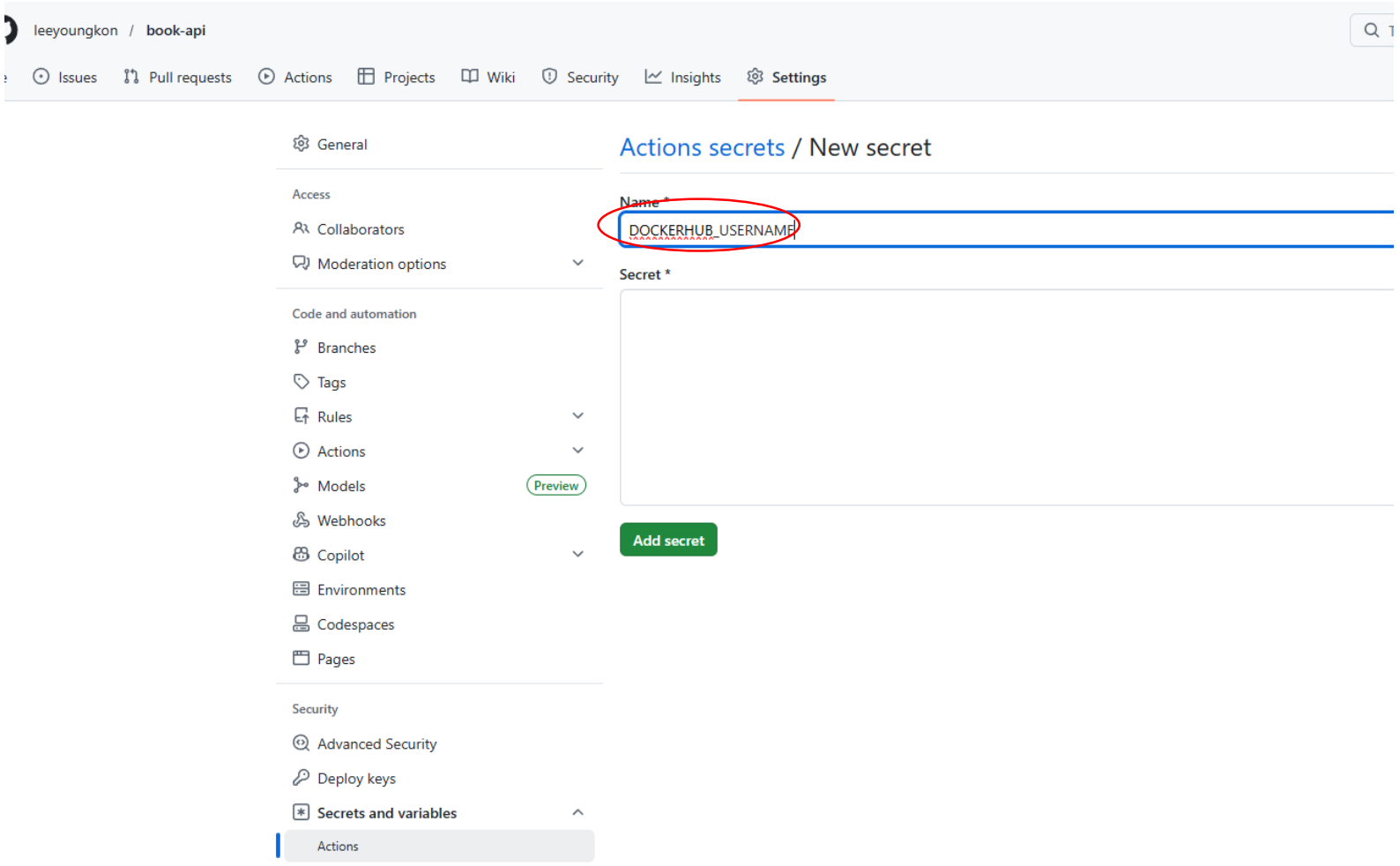
Preparation at Github: set secret for github access



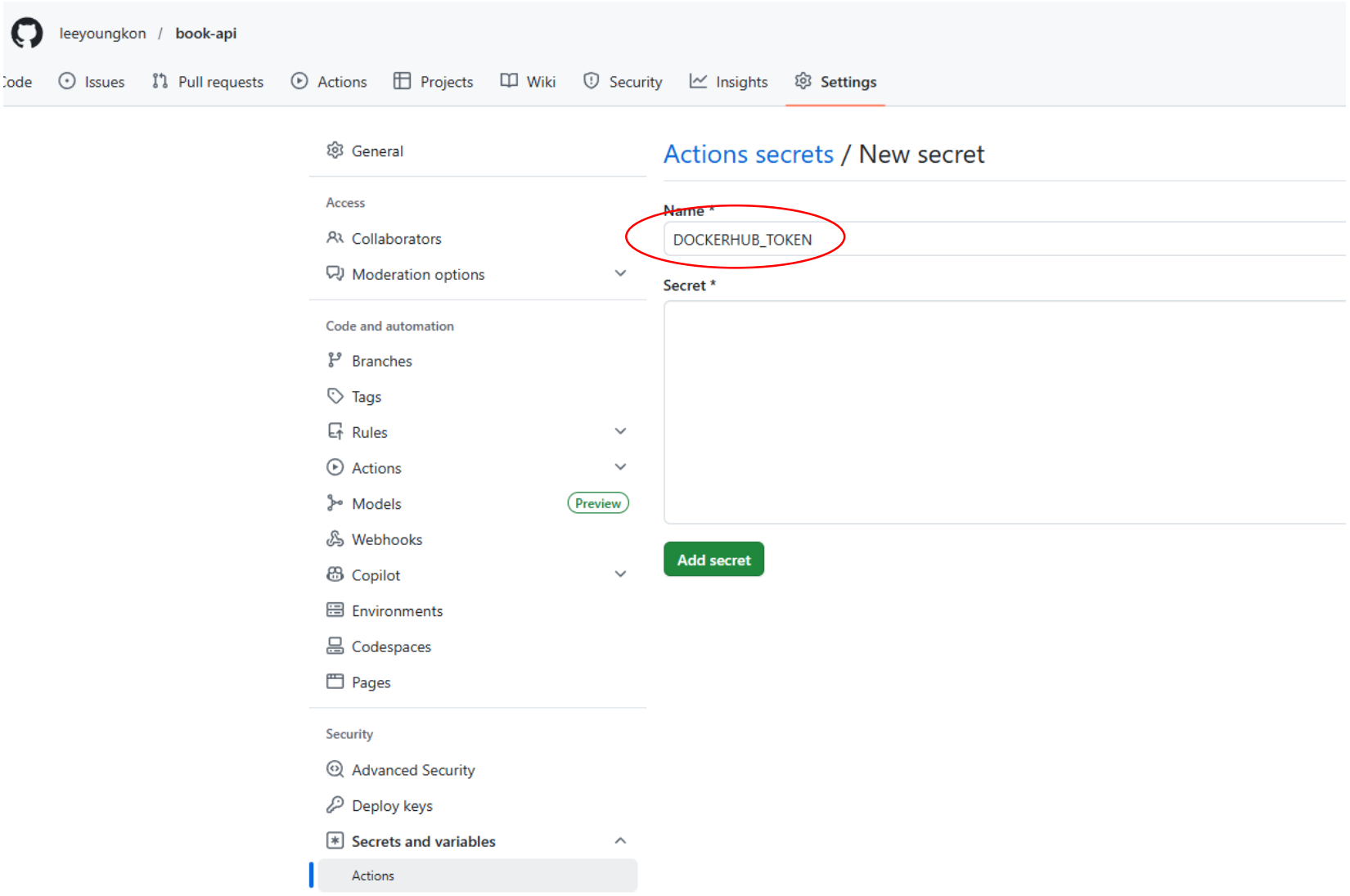
Preparation at Github: set secret for github access



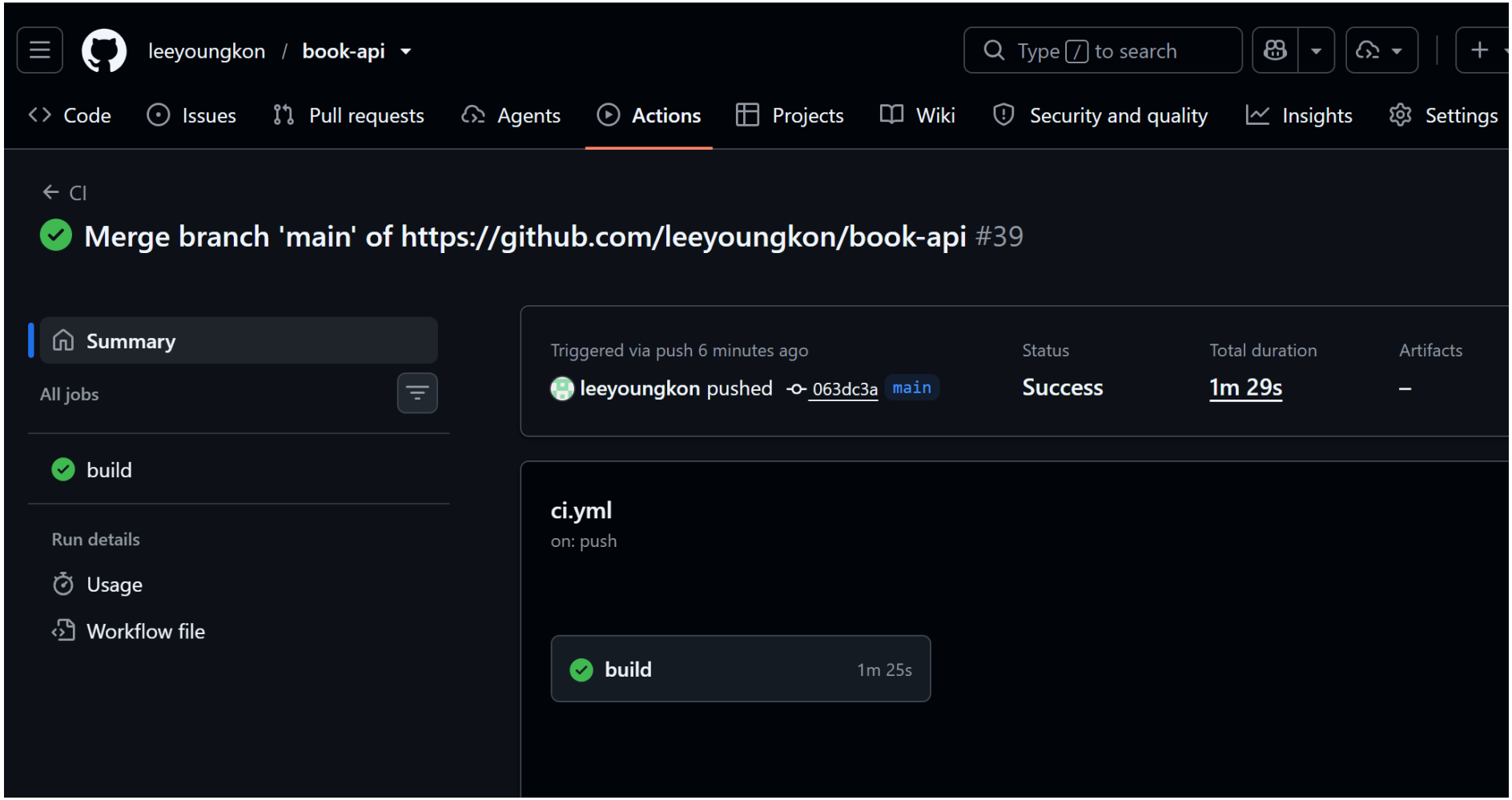
Preparation at Github: set secret for github access



Preparation at Github: set secret for github access



CI test for Book-api project by github hosted runner



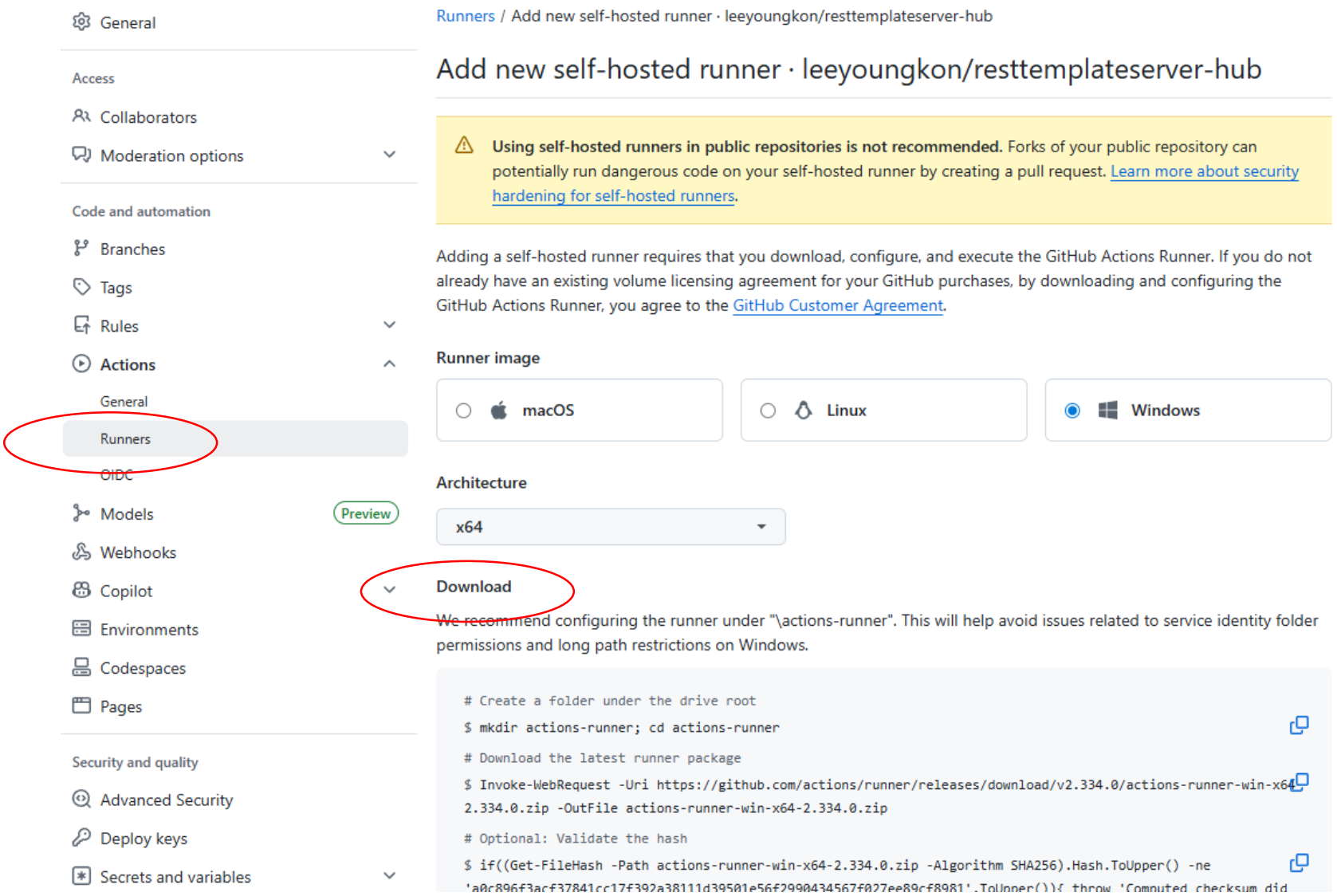
Preparation at Github: Determining where to run workflow

- GitHub Hosted Runner
  - Run on servers provided by GitHub
  - runs-on: ubuntu-latest
  - Flow: GitHub Push → Runner on the GitHub server → Build/Test/Push
  - No local PC required
- Self-hosted Runner
  - Run on my PC or on-premises server
  - runs-on: self-hosted
  - Flow: GitHub Push → GitHub Forward Tasks to My Runner → Run on My PC
  - Local Runner Required

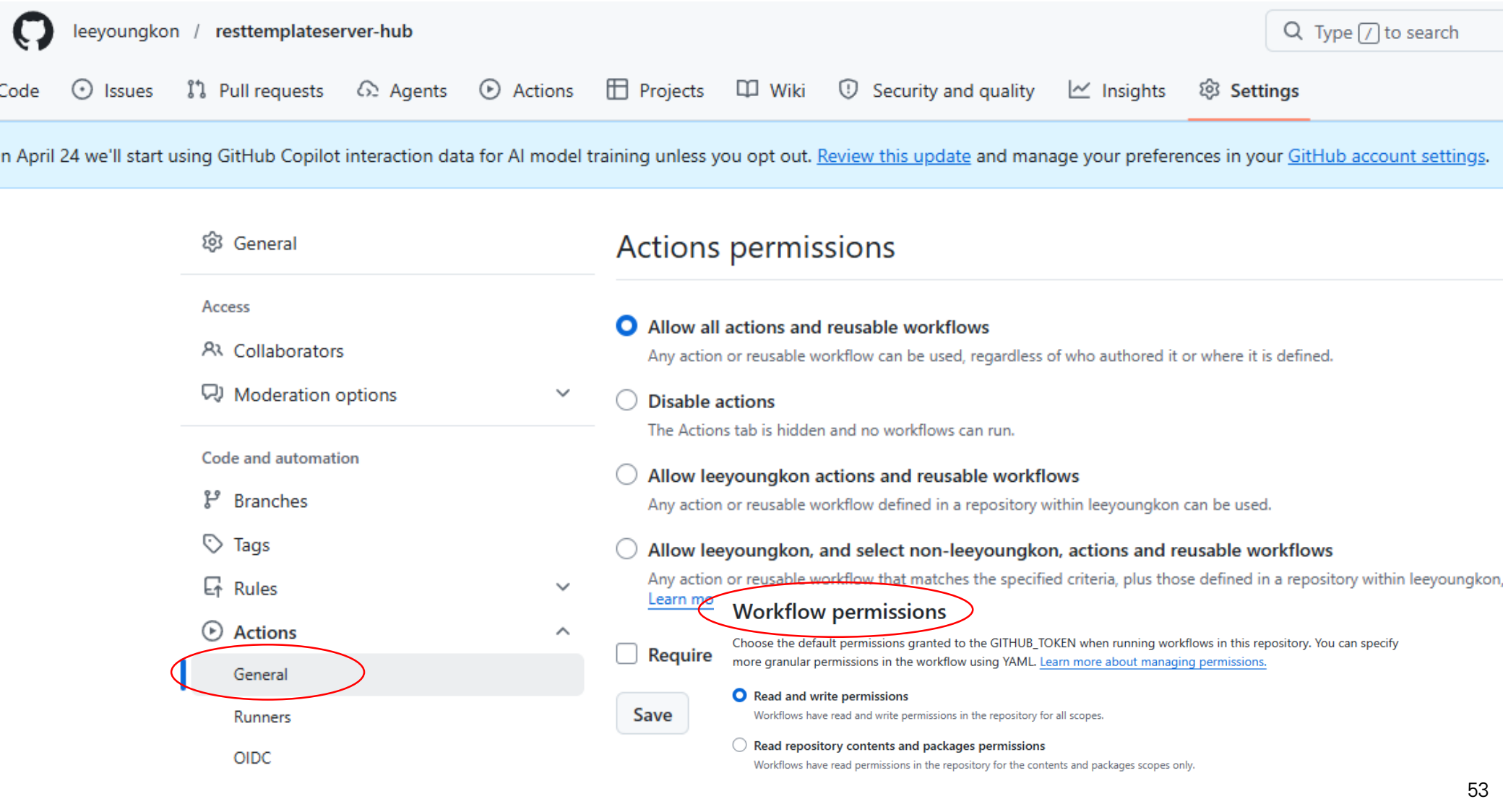
Ready at Github: If You Need a Self-Hosted Runner

- If you have resources that the GitHub server does not have direct access to:
  - In-house Harbour
  - Internal Kubernetes
  - Private network DB
  - Docker Desktop
  - On-premises infrastructure

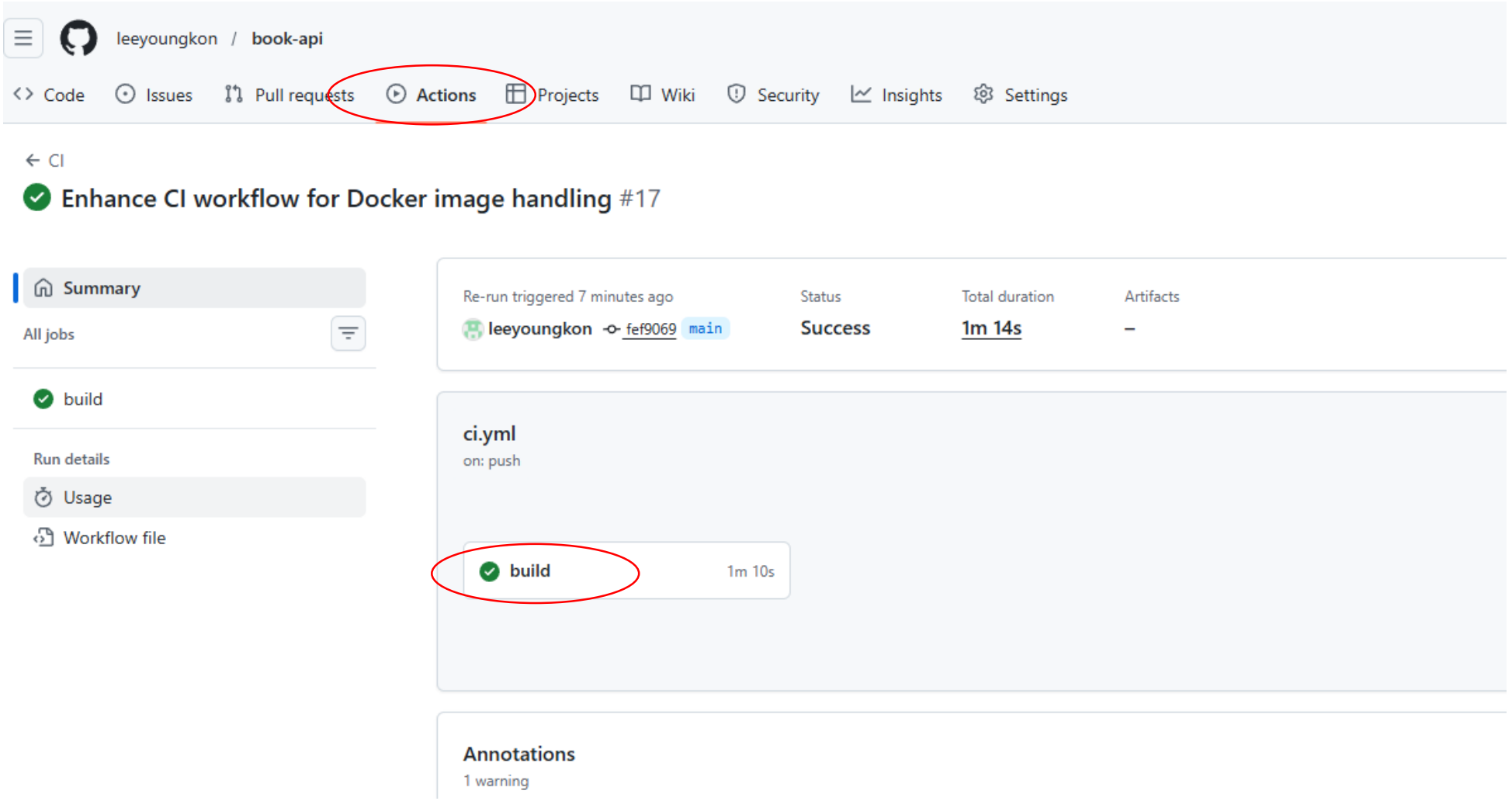
Preparation at Github: self-hosted runner register ⇒ execute cmd



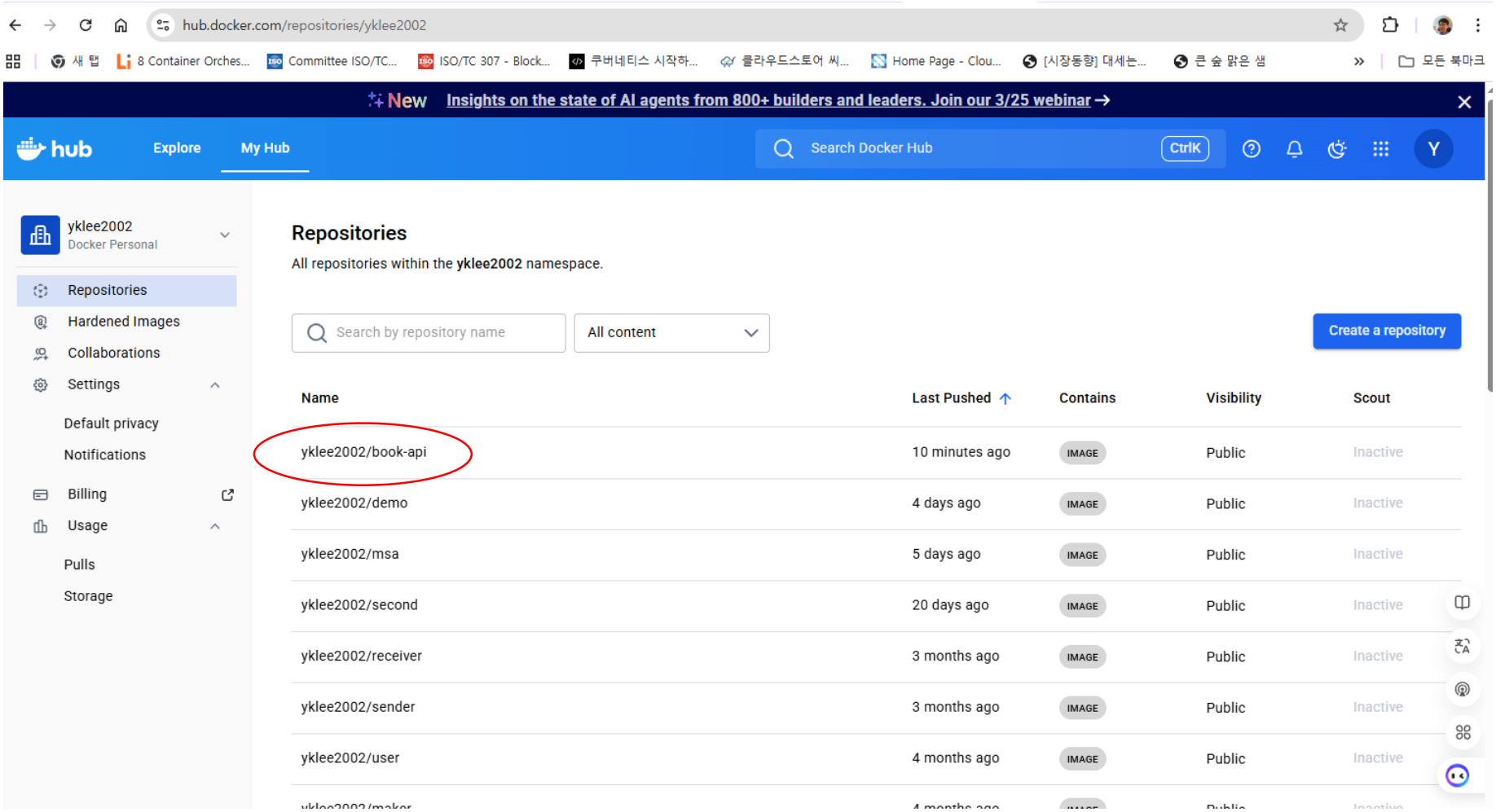
Preparation at Github: action ⇒ general ⇒ W/F permission grant



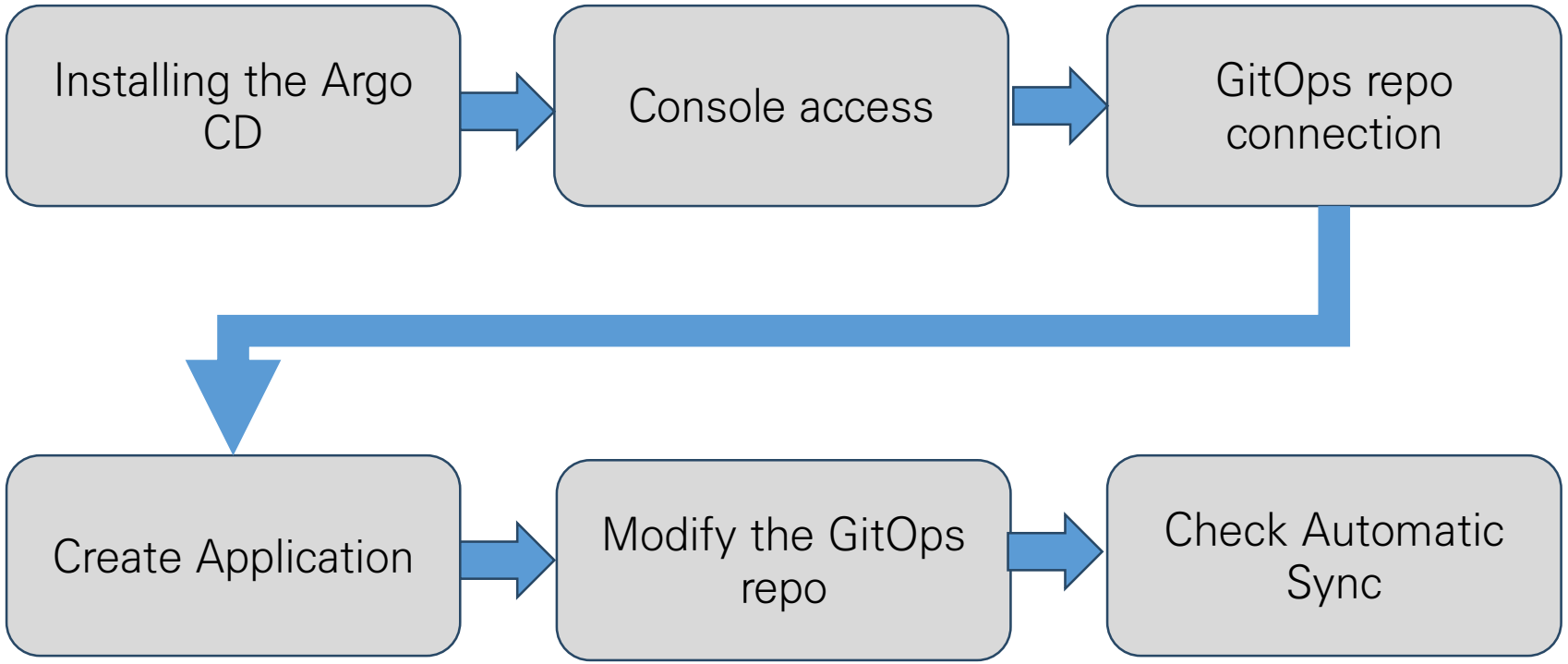
Execute CI workflow by actions-runner



Confirm docker image at docker hub



Process


Installing the Argo CD
Console access
GitOps repo connection
Create Application
Modify the GitOps repo
Check Automatic Sync

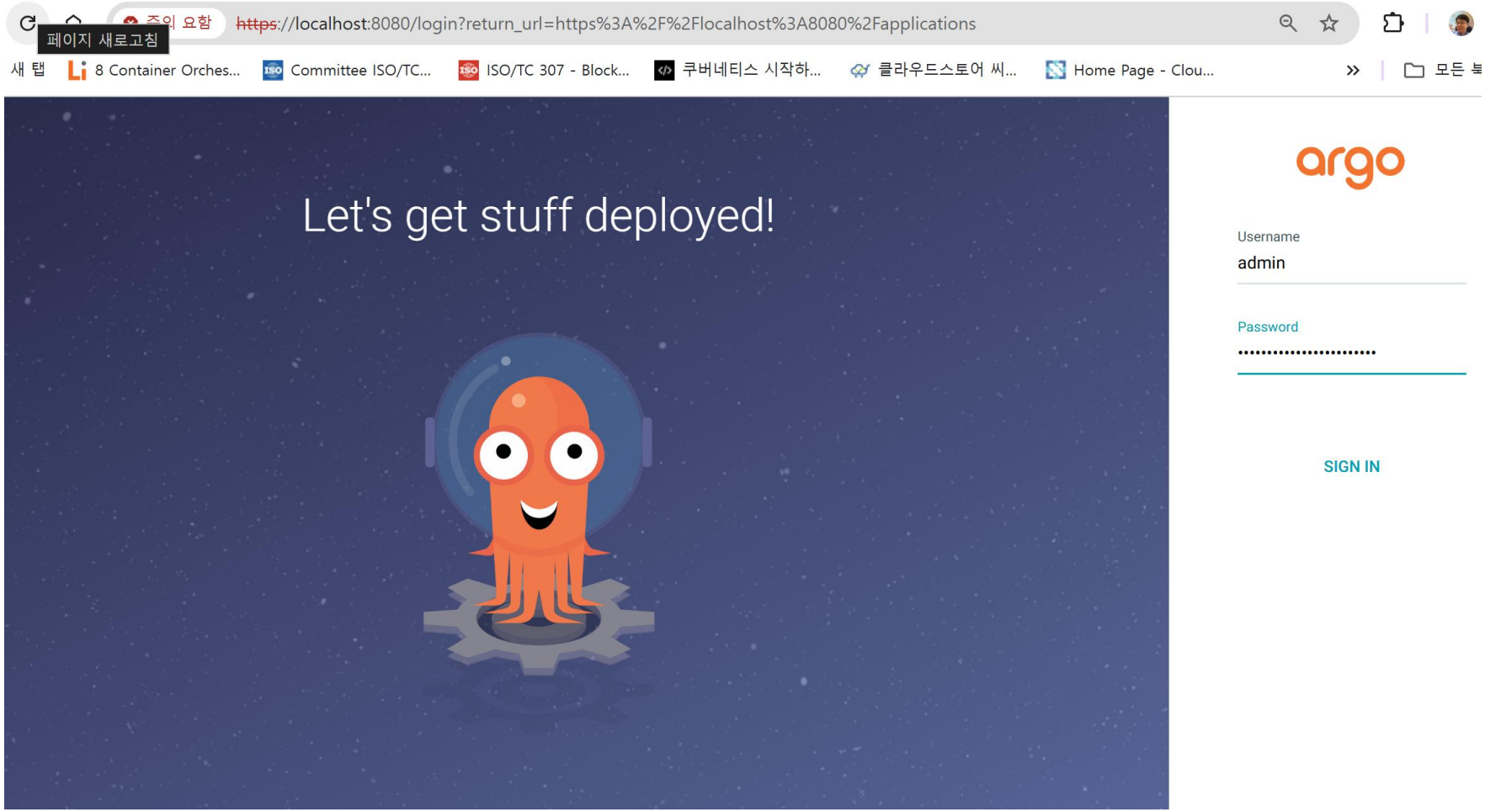
Check the context for ArgoCD installation

- kubectl config current-context //Check current context
- check kubectl get nodes //node
- kubectl config get-contexts // View context information
- kubectl cluster-info //View cluster information currently in use
- kubectl config view //Config view view
- convert kubectl config use-context default //context to default

Installing the Argo CD

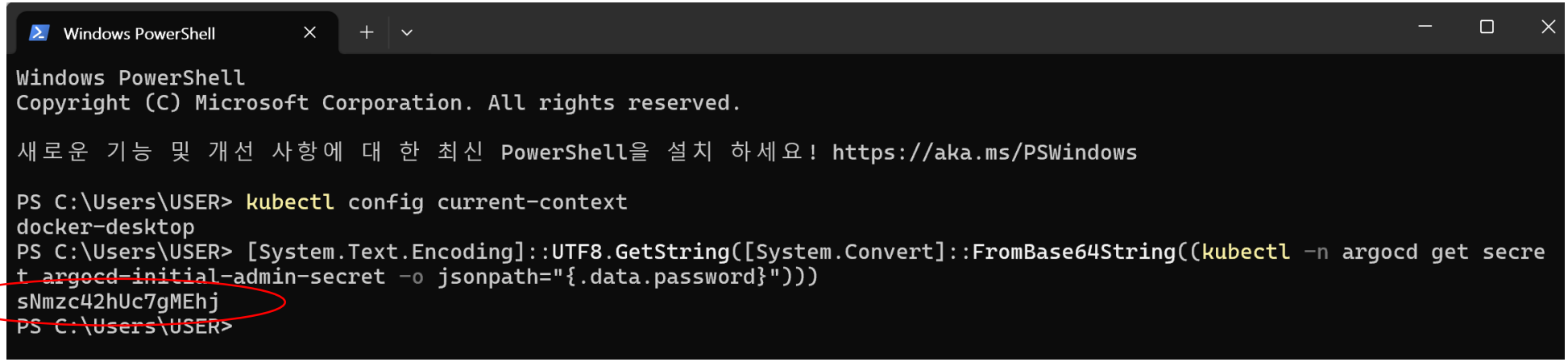
- kubectl create namespace argocd
- kubectl apply -n argocd --server-side --force-conflicts -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
- kubectl get pods -n argocd
- kubectl port-forward svc/argocd-server -n argocd 8080:443
- kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d && echo //linux server
- [System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String((kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}"))) //windows powershell

ArgoCD UI 접속: localhost:8080

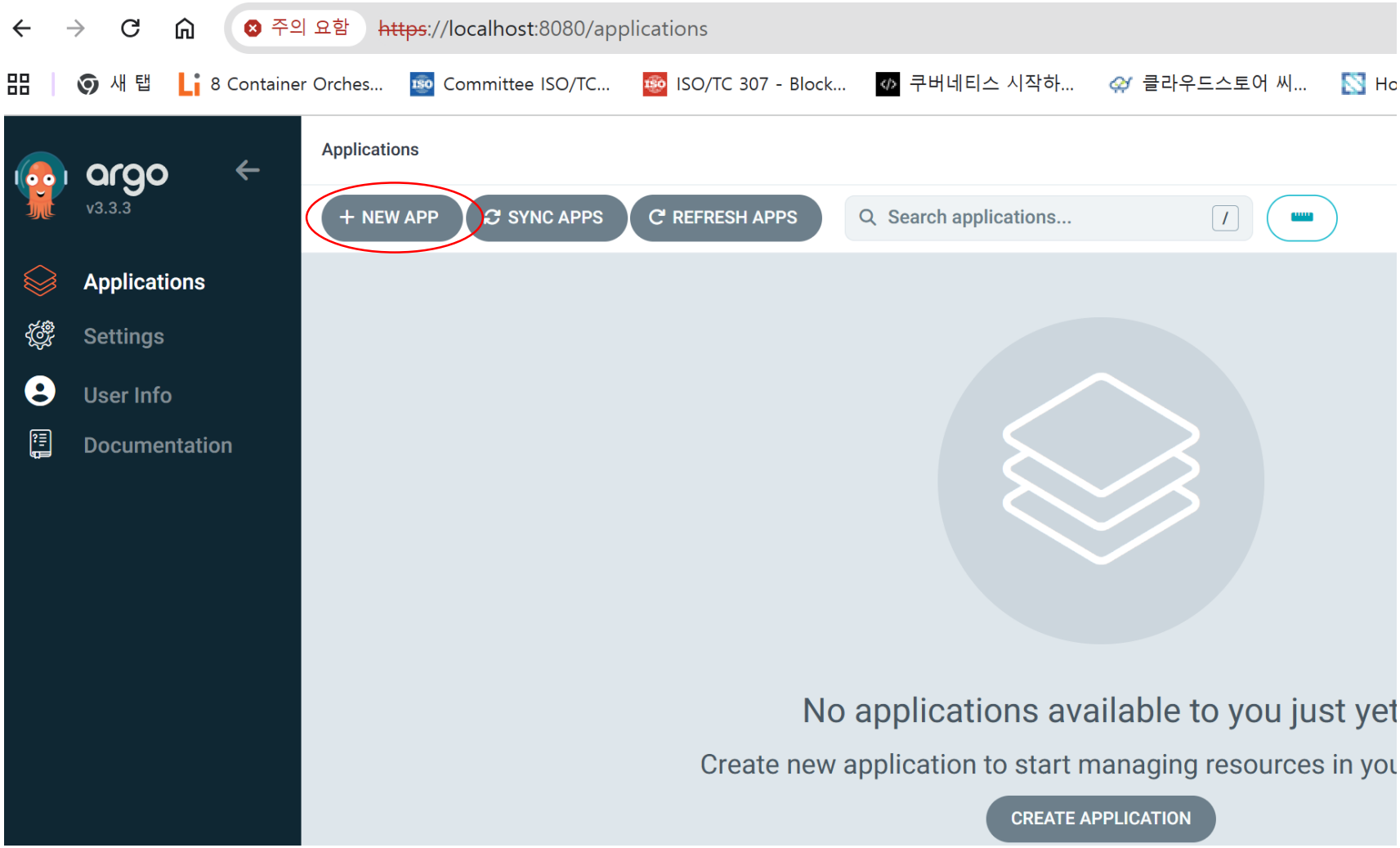


Log in to ArgoCD

- ID: admin
- Password: Enter the password generated by powershell as shown below



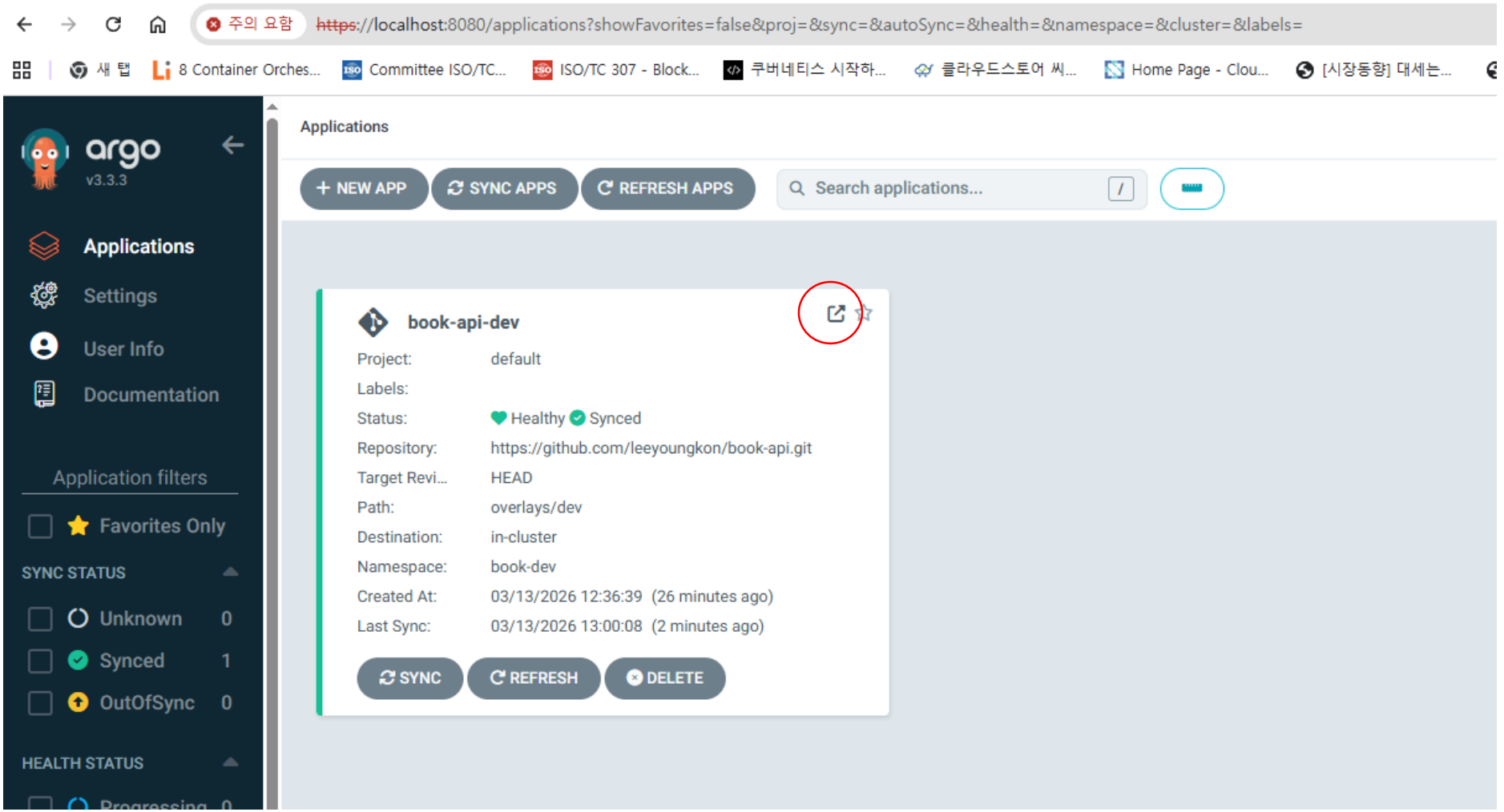
CD application process to developed apps



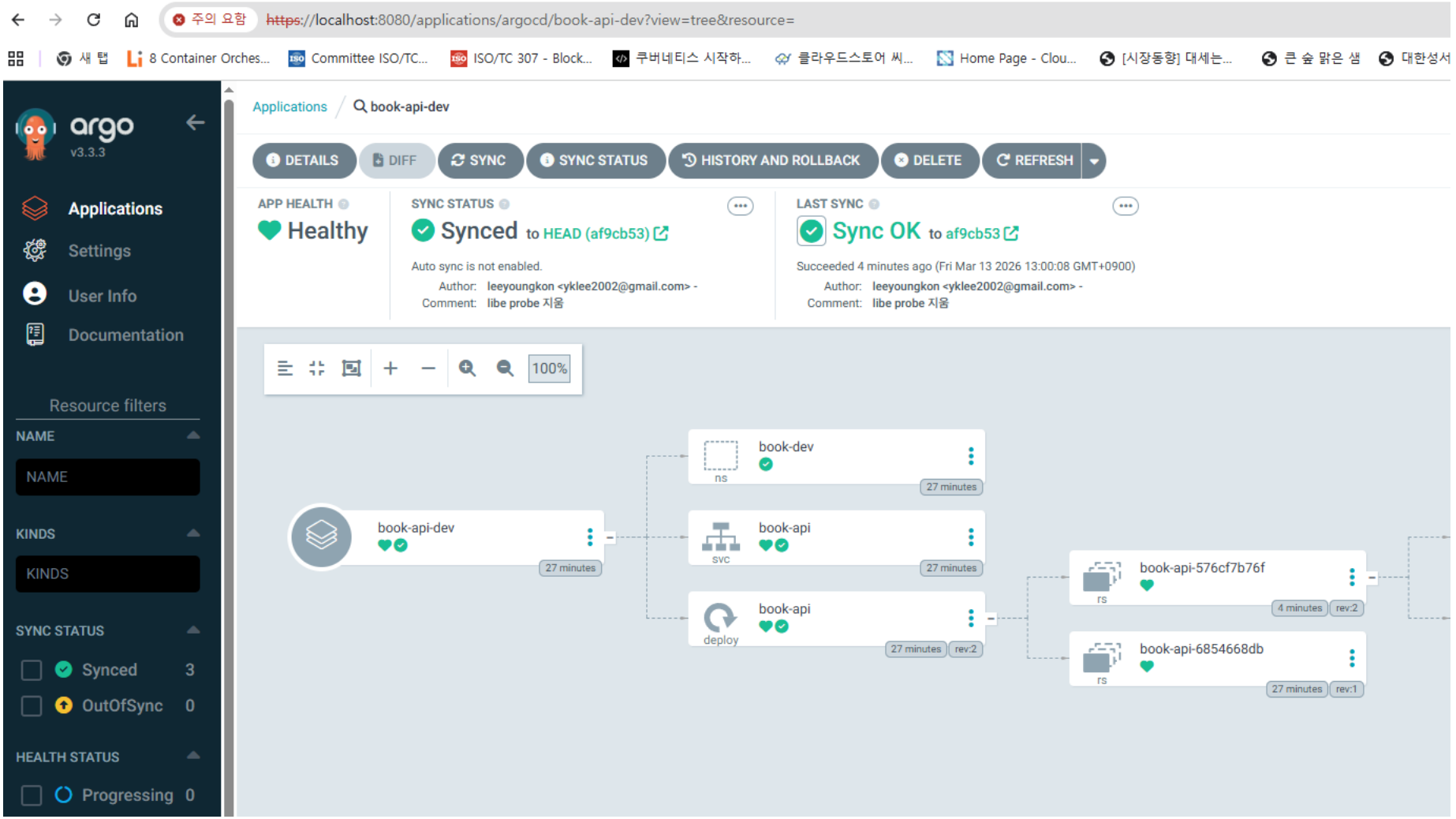
Create an Application from ArgoCD

- Name: book-api-dev
- Project: default
- Sync policy: manual (first)
- Repository URL: https://github.com/yourid/book-api.git
- Revision: HEAD
- Path: .
- Cluster URL: https://kubernetes.default.svc
- Namespace: default (or namespace you made)
- Press the create button at the end

Application sync on ArgoCD



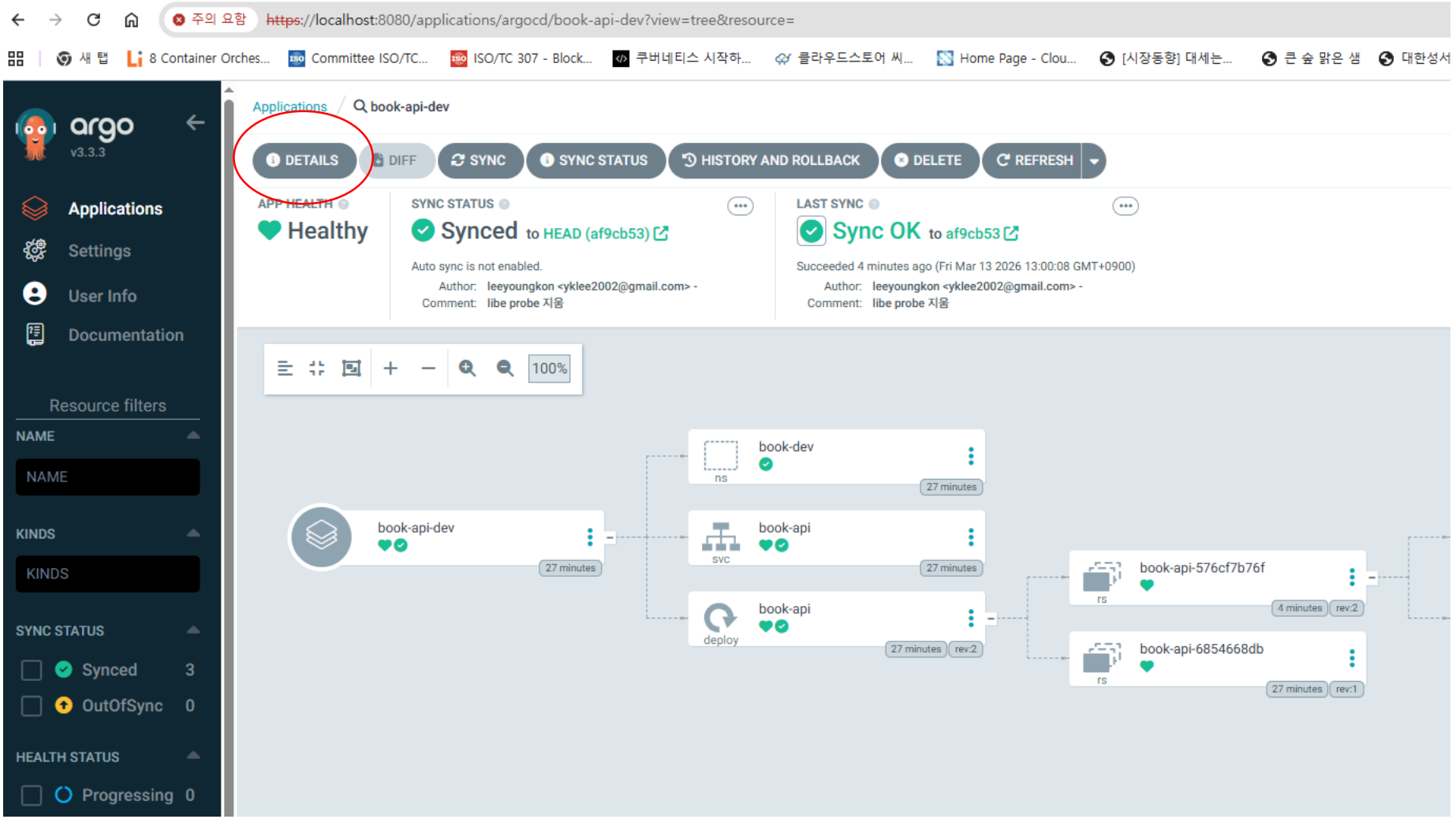
Verifying the Running Status on ArgoCD



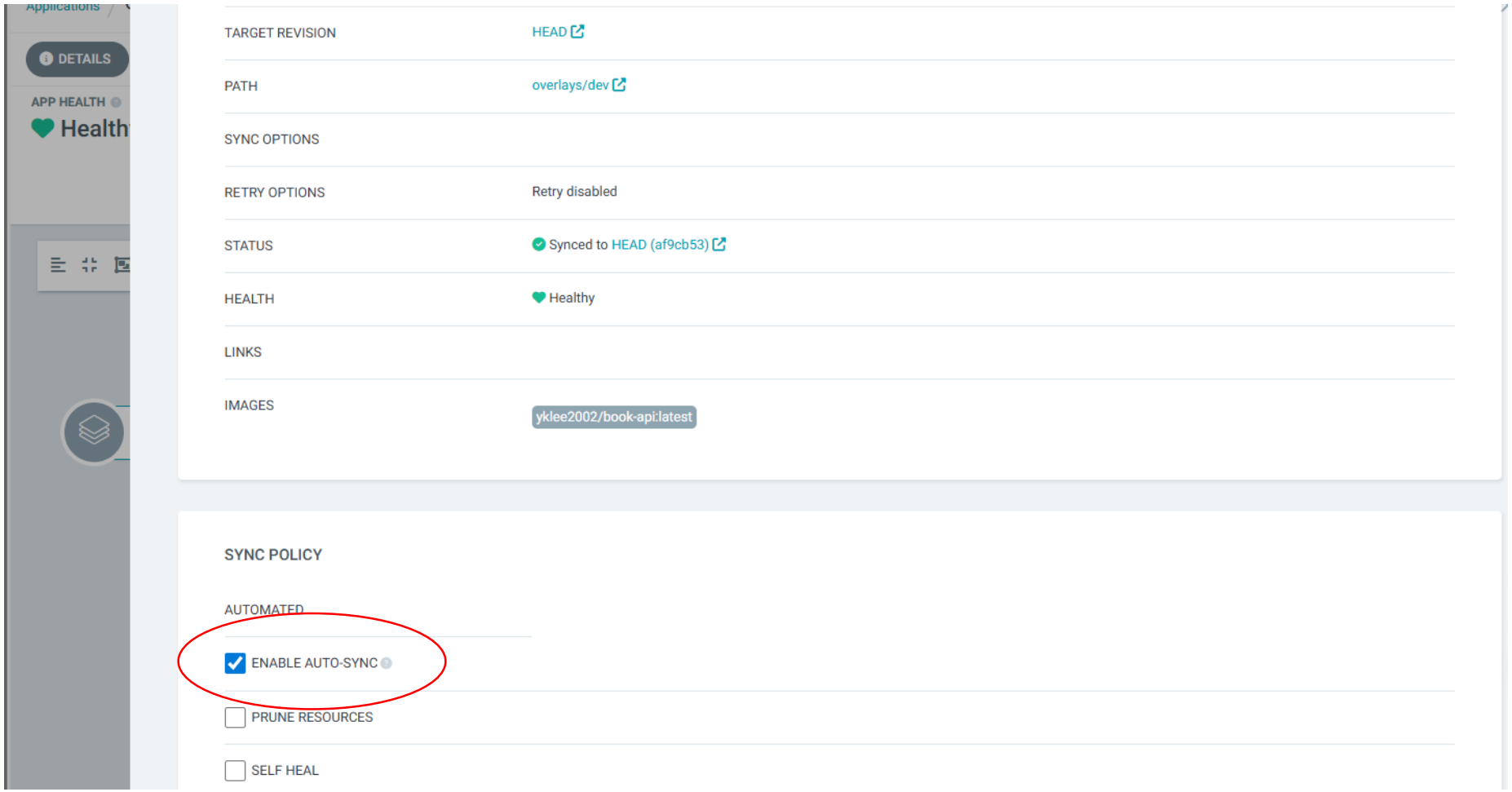
Also see in the command window

```
C:₩Users₩leeyoungkon>kubectl get all -n book-dev
NAME                            READY   STATUS    RESTARTS   AGE
pod/book-api-576cf7b76f-kk5m7   1/1     Running   0          10s
pod/book-api-576cf7b76f-qzq8p   1/1     Running   0          21s

NAME                 TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
service/book-api     ClusterIP   10.110.16.218   <none>        80/TCP    23m

NAME                          READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/book-api      2/2     2            2           23m

NAME                                     DESIRED   CURRENT   READY   AGE
replicaset.apps/book-api-576cf7b76f      2         2         2       21s
replicaset.apps/book-api-6854668db       0         0         0       23m
```

Auto sync settings on ArgoCD: click details

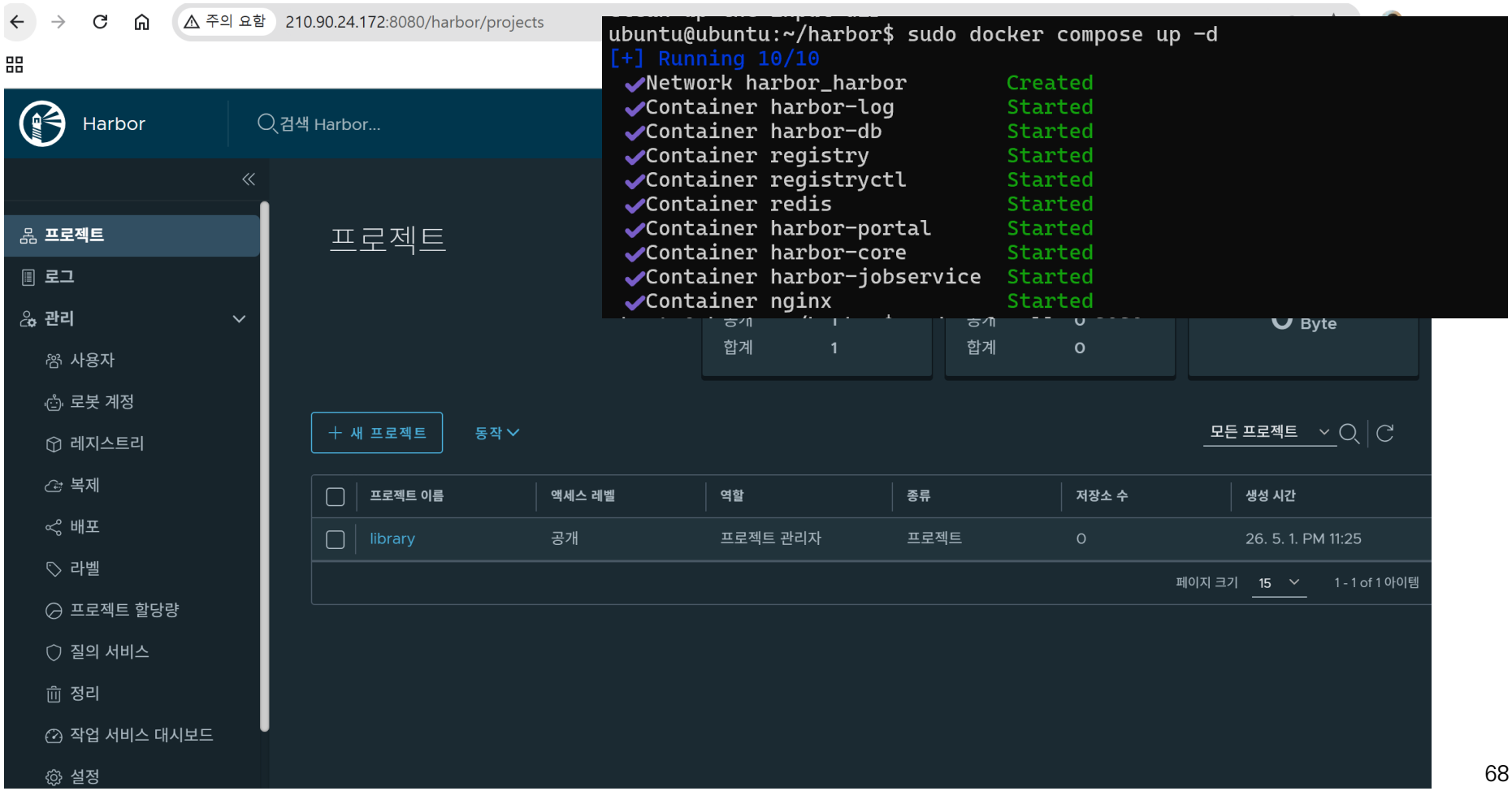


Auto sync setting in ArgoCD: Auto sync check in summary tab



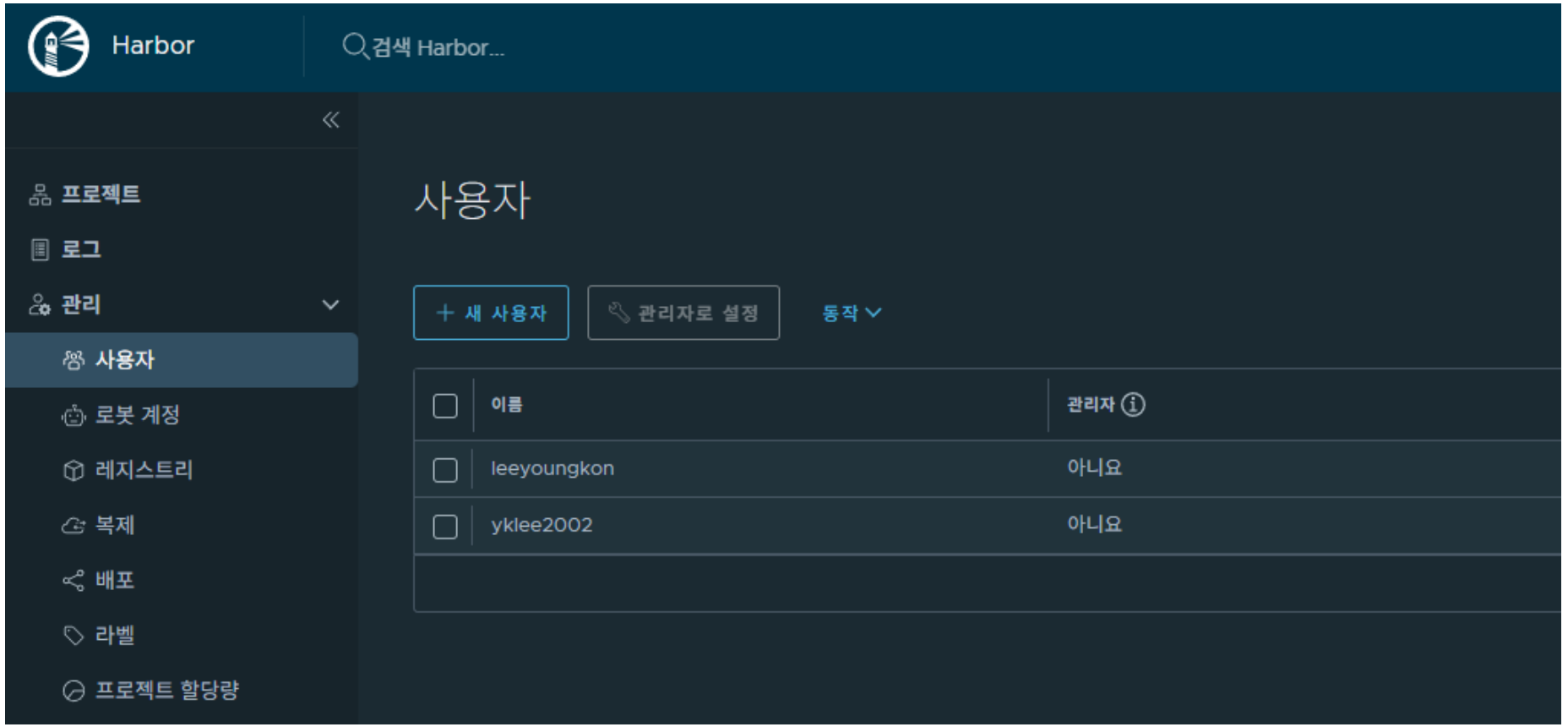
Docker Image Registry

- Login url: http://210.90.24.172:8080, id : admin, pwd : Harbor12345



Docker Image Registry

- After you create a project, you must register a user and register that user with the project
- Project Repository is automatically generated when you first push an image



CI.yaml (change with Harbour use)

```yaml
env:
  HARBOR_REGISTRY: 210.90.24.172:8080
  HARBOR_PROJECT: restproject
  STUDENT_ID: leeyoungkon
  IMAGE_NAME: resttemplateserver
```

repository name

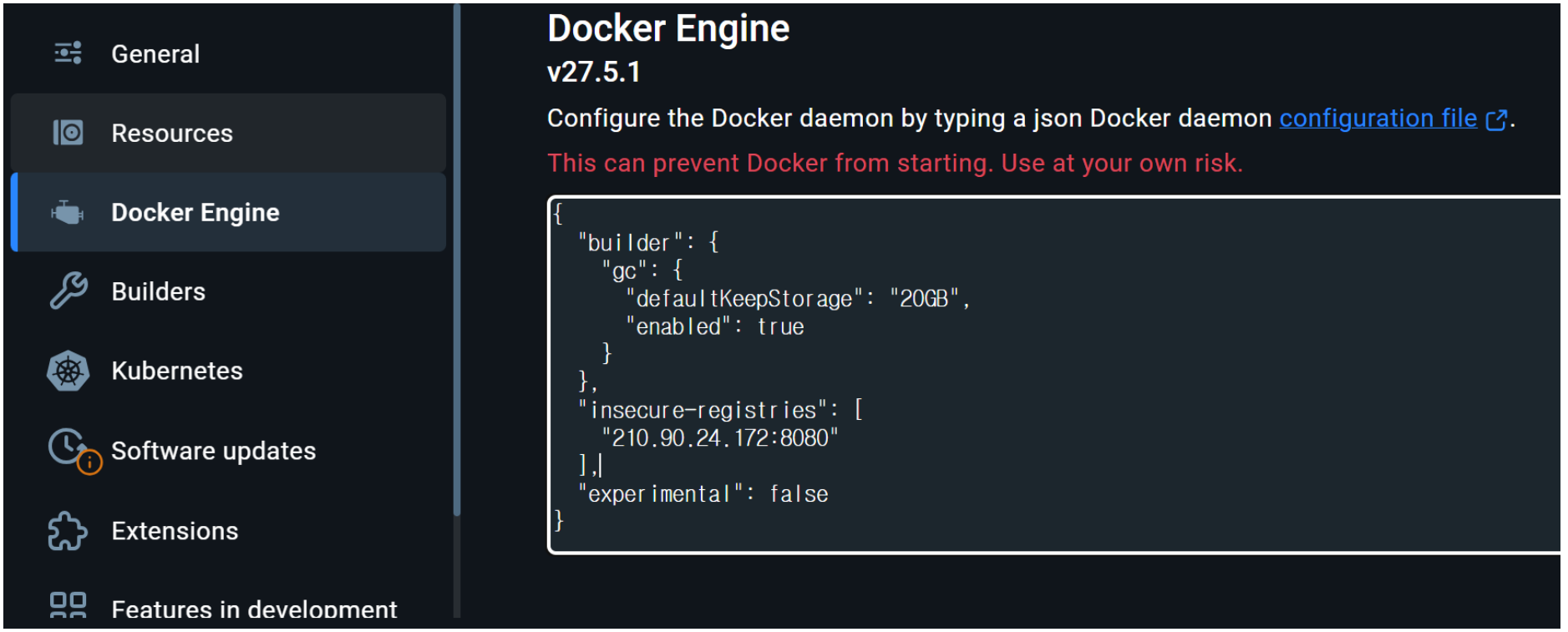
```yaml
  registry: ${{ env.HARBOR_REGISTRY }}
  username: ${{ secrets.HARBOR_USERNAME }}
  password: ${{ secrets.HARBOR_PASSWORD }}

name: Build and push Docker image
uses: docker/build-push-action@v5
with:
  context: .
  push: true
  tags: |
    ${{ env.HARBOR_REGISTRY }}/${{ env.HARBOR_PROJECT }}/${{ env.STUDENT_ID }}/${{
    ${{ env.HARBOR_REGISTRY }}/${{ env.HARBOR_PROJECT }}/${{ env.STUDENT_ID }}/${{
```

k8s.yaml change part

```
spec:
  containers:
    - name: resttemplateserver
      image: 210.90.24.172:8080/restproject/leeyoungkon/resttemplateserver:eddda1e8bf5d9538f
      imagePullPolicy: Always
      ports:
        - containerPort: 8080
```

[Harbor registry ip] [Project Name] [Repository Name] [Image Name: SHA Tag]

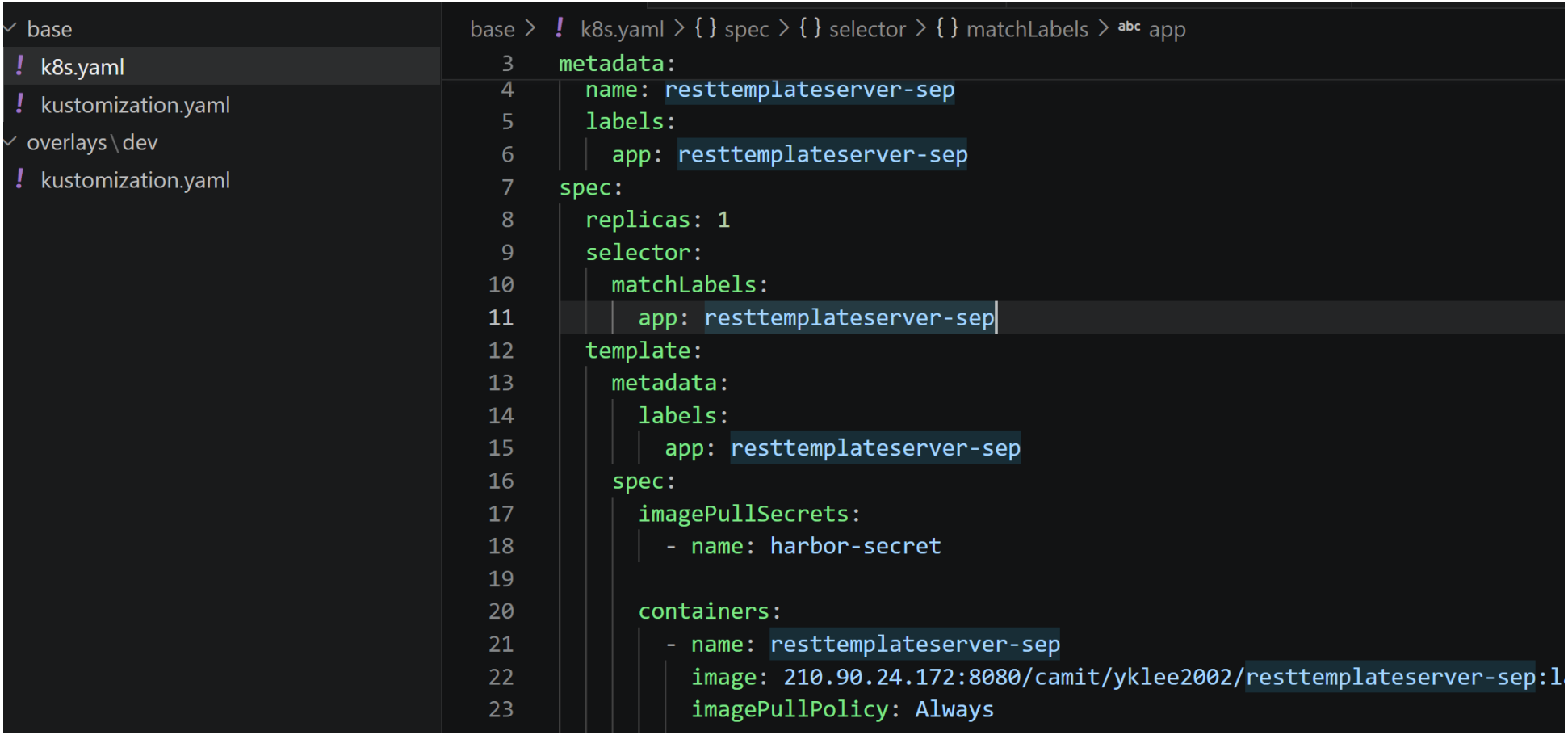
Add Insecure Registry Information to Docker Engine



To register Harbour as an insecure registry and access it to http

```
"insecure-registries": [
  "210.90.24.172:8080"
]
```

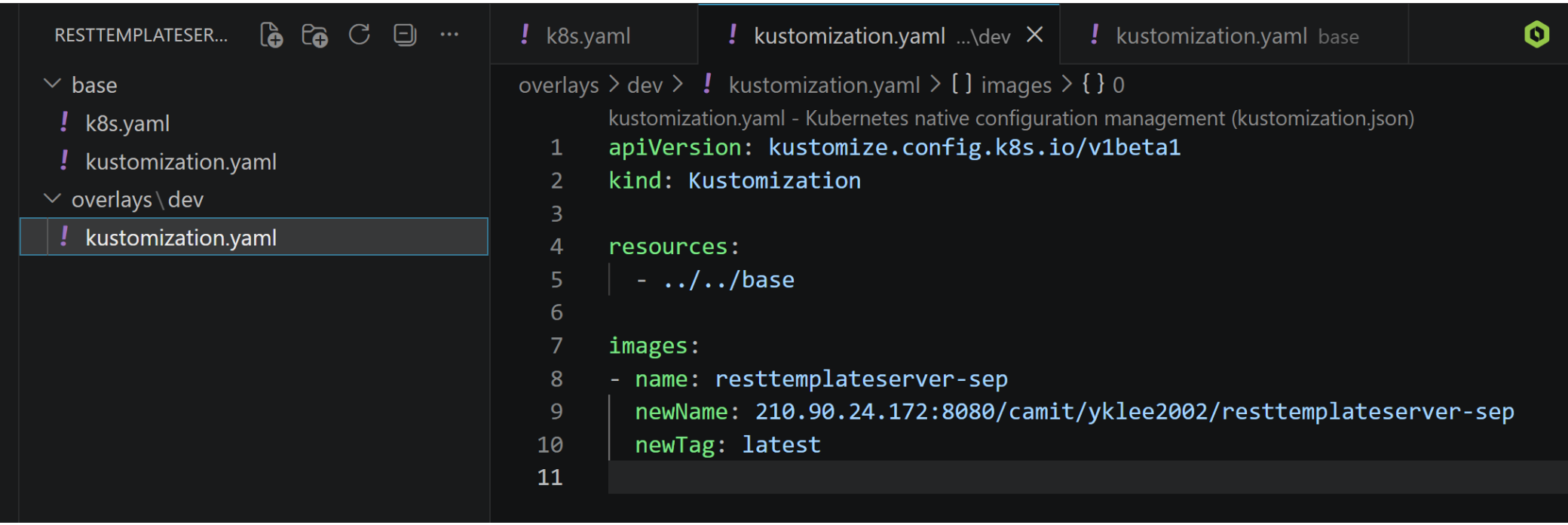
In k8s.yaml, harbor secret is required to get docker image in Harbor



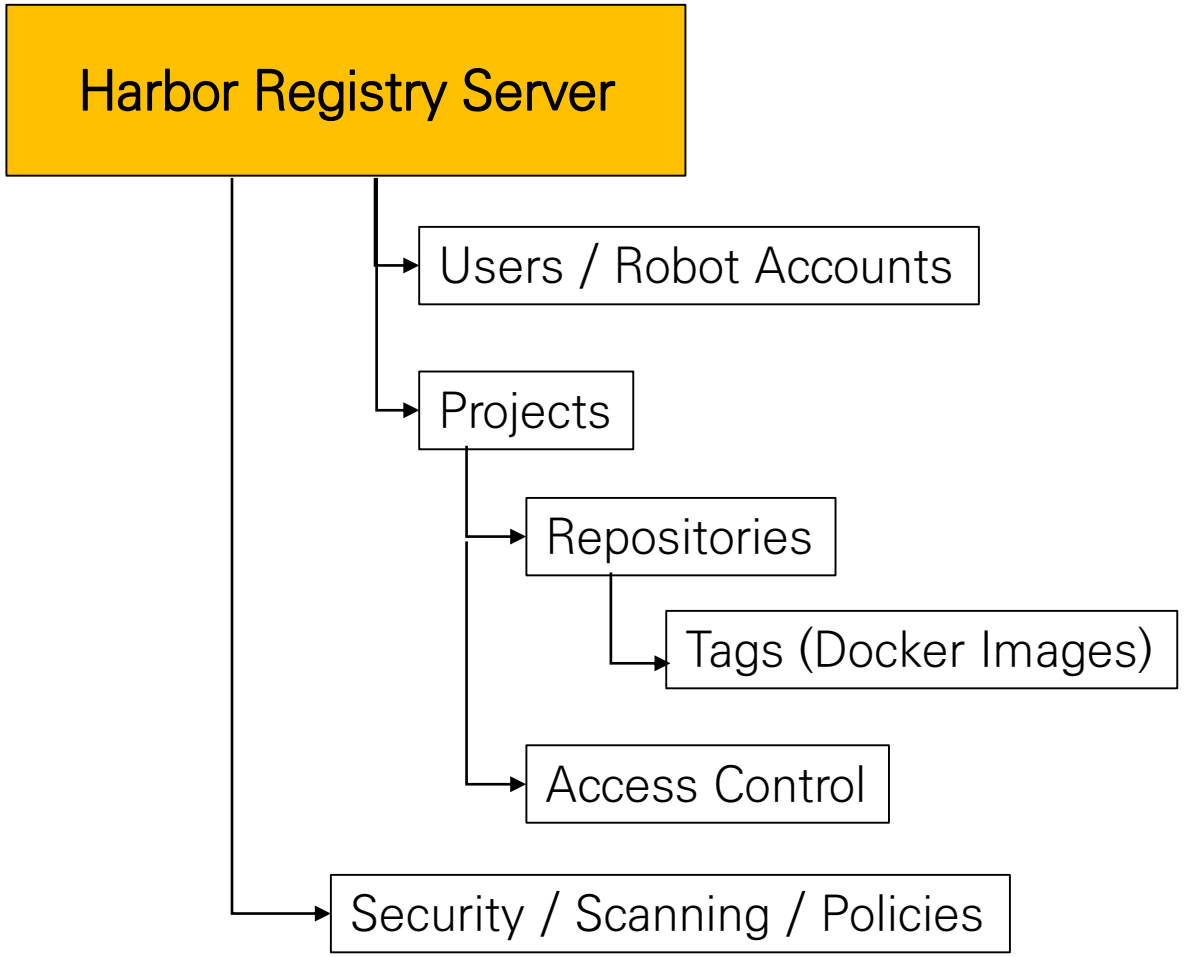
kubectl create secret docker-registry harbor-secret --docker-server=210.90.24.172:8080 --docker-username=yklee2002 --docker-password=Byby3845// --docker-email=yklee2002@gmail.com -n default

08 Harbor

In kustomization.yaml, resources and images should be included.



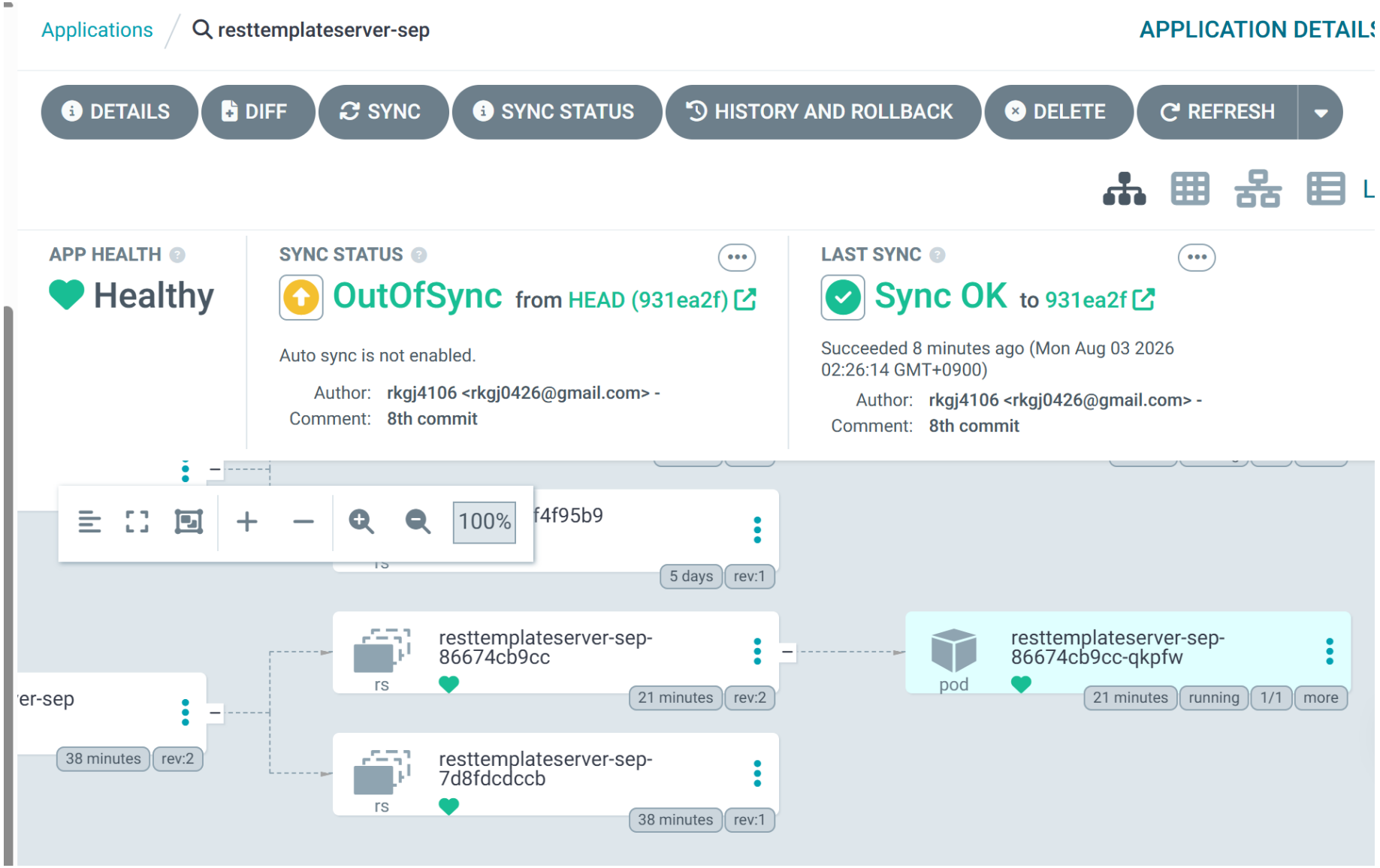
Harbor Configuration



Harbor Element Description

Element	Necessity	Description
Harbor server (Registry)	Essential	Full Storage
Project	Essential	push target
User/Robot Account	Essential	Certified.
Project Permissions	Essential	Allow push
GitHub Secrets	Essential	CD Authentication
Repository	Automatic generation	The first push

Harbor


Applications
resttemplateserver-sep
APPLICATION DETAILS
DETAILS
DIFF
SYNC
SYNC STATUS
HISTORY AND ROLLBACK
DELETE
REFRESH
APP HEALTH
Healthy
SYNC STATUS
OutOfSync from HEAD (931ea2f)
Auto sync is not enabled.
Author: rkgj4106 <rkgj0426@gmail.com> -
Comment: 8th commit
LAST SYNC
Sync OK to 931ea2f
Succeeded 8 minutes ago (Mon Aug 03 2026 02:26:14 GMT+0900)
Author: rkgj4106 <rkgj0426@gmail.com> -
Comment: 8th commit
100%
f4f95b9
5 days
rev:1
resttemplateserver-sep-86674cb9cc
rs
21 minutes
rev:2
resttemplateserver-sep-86674cb9cc-qkpfw
pod
21 minutes
running
1/1
more
er-sep
38 minutes
rev:2
resttemplateserver-sep-7d8fdcdccb
rs
38 minutes
rev:1

Thank You